

Konsep Website Invoice Kerajinan Tangan

<https://gemini.google.com/app/7d0358c947acb5a9>

User prompt: Saya mau buat web untuk invoice. Jadi untuk permohonan invoice dari awal transaksi sampai ke pengiriman dari invoice tersebut bisa send email, send WA, maupun di print. Nah, yang jadi permasalahan aku belum tahu nih konsep untuk websitenya seperti apa, untuk alurnya gimana. Jadi ini website, jadi ini jujur ini website khusus untuk handicraft, jadi merupakan kerajinan tangan, ya biasa rajut-rajutan gitu. Dengan nama perusahaan DynoBoo, jadi belum ada konsep gamabrannya nih, bisa kasih kan konsep rancangannya serta rancangan database nya tidak

Response: Halo! Konsep website pembuatan invoice untuk usaha kerajinan tangan (handicraft/rajutan) seperti **DynoBoo** sangat prospektif. Usaha rajut memiliki keunikan tersendiri: sering kali menggunakan sistem **Pre-Order (PO)**, butuh catatan kustomisasi (warna benang, karakter, ukuran), serta skema pembayaran bertahap (Down Payment / DP).

Berikut adalah rancangan konsep alur sistem, fitur utama, dan struktur database yang dirancang khusus untuk karakteristik bisnis **DynoBoo**.

1. Konsep Alur Kerja Sistem (Workflow)

Alur transaksi dirancang fleksibel untuk mengakomodasi barang *ready stock* maupun *custom PO*:

```

[Permintaan Masuk] —> [Buat Invoice (Draft/Pending)] —> [Kirim Invoice via WA/Email/Print]
                                     |
[Barang Dikirim] <— [Proses Produksi/Rajut] <— [Terima Pembayaran (DP / Lunas)]
  
```

1. Input Pesanan (Order Request):

- Admin memasukkan data pelanggan dan detail pesanan (termasuk *custom note* seperti warna/karakter rajutan).

2. Generasi Invoice:

- Sistem membuat invoice unik (contoh: INV/DNB/202607/001).
- Mengkalkulasi subtotal, ongkir, potongan harga (jika ada), dan status pembayaran (Unpaid / DP / Paid).

3. Pengiriman Invoice ke Pelanggan:

- WhatsApp:** Mengirimkan pesan teks otomatis berisi ringkasan tagihan beserta *link preview* atau attachment PDF.
- Email:** Mengirimkan email resmi berformat HTML rapi dengan lampiran PDF Invoice.
- Print:** Mencetak langsung invoice ke format cetak (A4, A5, atau Struk Thermal/Receipt).

4. Pembayaran & Update Status:

- Admin mencatat pembayaran masuk (bisa DP dulu atau langsung Lunas).
- Status invoice otomatis berubah (misal: *Partial/DP Paid* -> *Fully Paid*).

5. Penyelesaian & Pengiriman:

- Status pengerjaan rajutan diperbarui (*In Production* -> *Ready to Ship* -> *Shipped* dengan nomor resi).

2. Fitur Utama Website DynoBoo

- Dashboard Analytics:** Ringkasan total penjualan, status invoice (Belum Bayar, DP, Lunas), dan pesanan rajutan yang sedang berjalan.
- Manajemen Pelanggan (CRM Sederhana):** Menyimpan riwayat belanja, nomor WhatsApp, dan alamat pengiriman.
- Invoice Builder:**
 - Pilihan jenis pesanan: *Ready Stock* vs *Custom PO*.
 - Catatan khusus tiap item (contoh: "*Benang Milk Cotton Warna Pink Pastel, Pakai Gantungan Kunci*").
 - Opsi Pembayaran DP (%) atau Pelunasan.
- Multi-Channel Dispatcher:**
 - Tombol "**Send via WhatsApp**" (Integrasi WA Gateway).
 - Tombol "**Send via Email**" (Integrasi SMTP).
 - Tombol "**Print / Download PDF**".

3. Rancangan Database (Relational Database)

Berikut adalah struktur tabel yang disarankan (menggunakan MySQL/PostgreSQL):

a. Tabel customers (Data Pelanggan)

Field Name	Type	Description
id (PK)	INT / UUID	ID Unik Pelanggan
name	VARCHAR(100)	Nama Lengkap
phone	VARCHAR(20)	Nomor WhatsApp (format: 628xxx)
email	VARCHAR(100)	Email Pelanggan
address	TEXT	Alamat Lengkap Pengiriman

b. Tabel products (Katalog Produk Rajut)

Field Name	Type	Description
id (PK)	INT / UUID	ID Unik Produk
name	VARCHAR(150)	Nama Produk (misal: Boneka Amigurumi Dino)
category	ENUM	ready_stock, custom_po
price	DECIMAL(12,2)	Harga Acuan/Dasar
description	TEXT	Deskripsi Produk

c. Tabel invoices (Header Invoice)

Field Name	Type	Description
id (PK)	INT / UUID	ID Unik Invoice
invoice_number	VARCHAR(50)	Kode Unik (misal: INV/DNB/202607/001)
customer_id (FK)	INT / UUID	Relasi ke tabel customers
issue_date	DATE	Tanggal Invoice Dibuat
due_date	DATE	Tanggal Jatuh Tempo
subtotal	DECIMAL(12,2)	Total harga barang
shipping_cost	DECIMAL(12,2)	Biaya Pengiriman
discount	DECIMAL(12,2)	Potongan Harga
total_amount	DECIMAL(12,2)	Total Keseluruhan
status	ENUM	draft, unpaid, partially_paid, paid, cancelled
order_status	ENUM	queued, in_production, ready, shipped
notes	TEXT	Catatan umum/Instruksi khusus pesanan

d. Tabel invoice_items (Detail Barang Dalam Invoice)

Field Name	Type	Description
id (PK)	INT / UUID	ID Unik Item
invoice_id (FK)	INT / UUID	Relasi ke tabel invoices
product_id (FK)	INT / UUID	Relasi ke tabel products (opsional)
item_name	VARCHAR(150)	Nama Barang saat transaksi
qty	INT	Jumlah Pesanan
price	DECIMAL(12,2)	Harga Satuan
custom_notes	TEXT	Detail kustomisasi (misal: ukuran, opsi warna benang)

e. Tabel payments (Riwayat Pembayaran / DP)

Field Name	Type	Description
id (PK)	INT / UUID	ID Unik Pembayaran
invoice_id (FK)	INT / UUID	Relasi ke tabel invoices
payment_date	DATETIME	Waktu Pembayaran Ditentukan
amount_paid	DECIMAL(12,2)	Nominal Pembayaran (DP / Pelunasan)
payment_method	VARCHAR(50)	Transfer BCA, QRIS, Cash, dll.
proof_file	VARCHAR(255)	Path/URL Foto Bukti Transfer

4. Rekomendasi Integrasi Fitur Pengiriman

1. WhatsApp Integration

- Gunakan layanan **WA Gateway / API** (seperti Fonnte, Wablas, atau WooSender).
- Saat tombol "Kirim WA" diklik, sistem menyusun pesan otomatis berisi perincian ringkas beserta link PDF Invoice.

2. Email Integration

- Gunakan layanan seperti **Resend**, **SendGrid**, atau SMTP bawaan hosting/Gmail.
- Template email menggunakan HTML responsif dengan tombol cetak dan attachment PDF terlampir.

3. Print & PDF Generation

- Gunakan library pembuat PDF di sisi server (seperti dompdf untuk PHP, atau Puppeteer/pdftkit untuk Node.js) agar tampilan cetak rapi, konsisten, dan berukuran standar (A5/A4/Thermal).

Apakah sistem invoice DynoBoo ini rencananya hanya akan dikelola internal oleh Admin/Pemilik toko, atau Kamu juga ingin menyediakan halaman portal di mana pelanggan bisa login dan melihat status pesanan rajutan mereka secara langsung?

User prompt: Invoicennya: Status Pembayaran ada 3 kategori yaitu : Lunas - Hijau DP - Kuning Belum Lunas - Merah Nama Barang (Decsripiton) ==> Disini mau ada database terpisah jadi ada database nama Produk/event (soalnya baisanya ada kegiatan workshop juga) and the jenis Produk/event jadi nanti bisa diatur di stock barang untuk jenis jenis produk yang ada biar fleksibel gitu

Response: Perubahan dan penambahan konsep ini sangat tepat! Memisahkan produk kerajinan (fisik) dan event workshop (jasa/tiket) akan membuat sistem DynoBoo jauh lebih rapi, terstruktur, dan fleksibel untuk perkembangan bisnis ke depan.

Berikut adalah penyesuaian konsep dan rancangan database terbaru yang mengakomodasi **Indikator Warna Status** serta **Pemisahan Master Data Produk & Event (berserta Stok/Kuota)**.

1. Penyesuaian Status Pembayaran & Indikator Warna

Sistem akan menggunakan logika kalkulasi otomatis atau input manual dengan indikator visual sebagai berikut:

Status	Warna Indikator	Kondisi / Logika Sistem
Lunas	● Hijau (#28A745)	Total Pembayaran Terbayar $\geq$ Total Tagihan Invoice.
DP	● Kuning (#FFC107)	Pembayaran Terbayar > 0 tetapi $<$ Total Tagihan Invoice.
Belum Lunas	● Merah (#DC3545)	Total Pembayaran Terbayar $= 0$ (Belum ada uang masuk).

Tampilan pada Invoice/PDF & Website: Status ini akan muncul dalam bentuk *Badge/Pill Tag* berwarna mencolok di bagian atas invoice agar pelanggan maupun admin langsung dapat mengidentifikasinya dalam sekali lihat.

2. Rancangan Database Terbaru (Master Produk & Event)

Untuk memisahkan antara **Produk Rajutan** dan **Event Workshop**, kita akan membuat struktur relasi antara:

- Jenis/Kategori** (item_types)
- Master Item/Produk/Event** (items)
- Detail Stok & Kuota**

[item_types / Jenis] — (1 to N) —> [items / Master Barang & Event] — (1 to N) —> [invoice_items]

A. Tabel item_types (Jenis Produk & Event)

Tabel ini berfungsi mengelompokkan jenis jualan DynoBoo agar stok dan laporan keuangan bisa difilter per jenis.

Field Name	Type	Description	Contoh Isi Data
id (PK)	INT / UUID	ID Jenis	1
name	VARCHAR(100)	Nama Jenis	"Handicraft Rajut", "Kit/Bahan Rajut", "Workshop Offline", "Workshop Online"
is_event	BOOLEAN	Penanda Event atau Barang Fisik	false (untuk Rajutan), true (untuk Workshop)

B. Tabel items (Master Produk / Event)

Tabel gabungan untuk semua item yang dijual (baik fisik maupun tiket workshop) agar mudah dipanggil di Form Invoice.

Field Name	Type	Description
id (PK)	INT / UUID	ID Unik Item
item_type_id (FK)	INT / UUID	Relasi ke item_types.id
code	VARCHAR(50)	Kode Item (misal: RJT-001, WS-2026-01)
name	VARCHAR(150)	Nama Produk / Nama Workshop
price	DECIMAL(12,2)	Harga Satuan / Biaya Pendaftaran
stock_or_quota	INT	Stok barang (jika fisik) atau Kuota Peserta (jika workshop)
track_stock	BOOLEAN	true (potong stok otomatis), false (misal untuk PO kustom tanpa batas)
event_date	DATETIME	Tanggal & Waktu Pelaksanaan (<i>Khusus Jenis Event/Workshop, nullable</i>)
event_location	VARCHAR(255)	Lokasi / Link Zoom (<i>Khusus Jenis Event/Workshop, nullable</i>)
is_active	BOOLEAN	Status item masih dijual / dibuka atau tidak

C. Pembaruan Tabel invoices & invoice_items

Tabel invoices (Header)

```
- id (PK)
- invoice_number
- customer_id (FK)
- issue_date
- due_date
- subtotal
- shipping_cost
- discount
- total_amount
- amount_paid      <-- TOTAL UANG YANG SUDAH MASUK
- payment_status   <-- ENUM: ('LUNAS', 'DP', 'BELUM_LUNAS')
- order_status     <-- ENUM: ('DRAFT', 'PROSES_RAJT', 'SIAP_KIRIM', 'SELESAI')
```

Tabel invoice_items (Detail Barang yang Dibeli)

```
- id (PK)
- invoice_id (FK)
- item_id (FK)      <-- Relasi ke tabel master 'items'
- item_name         <-- Disimpan ulang sebagai histori jika nama di master berubah
- item_type_name    <-- Disimpan ulang (misal: "Workshop Offline" / "Handicraft Rajut")
- qty              <-- Jumlah pcs / Jumlah tiket peserta
- price            <-- Harga per pcs/tiket saat transaksi
- custom_notes     <-- Bebas isi catatan:
                    • Untuk Rajutan: "Warna benang Pink, pakai Gantungan"
                    • Untuk Workshop: "Nama Peserta: Budi (Kaos Size L)"
- subtotal         <-- (qty * price)
```

3. Cara Kerja di Form Pembuatan Invoice (User Experience)

Saat Admin DynoBoo membuat invoice baru di website:

1. **Pilih Pelanggan** (atau isi data pelanggan baru).
2. **Tambah Item Tagihan:**
 - Dropdown 1: **Pilih Jenis** -> Filter opsi (Handicraft Rajut, Workshop Offline, dll).

- Dropdown 2: **Pilih Nama Barang/Event** -> Menampilkan daftar sesuai Jenis.
 - *Contoh A:* Admin pilih "**Handicraft Rajut**" -> Muncul "**Boneka Amigurumi Dino**" (Stok Tersedia: 5).
 - *Contoh B:* Admin pilih "**Workshop Offline**" -> Muncul "**Workshop Rajut Pemula - 15 Aug**" (Sisa Kuota: 3 kursi).
- **Qty:** Masukkan jumlah barang/tiket.
- **Catatan Khusus (Custom Note):** Kolom teks bebas untuk detail kustomisasi rajutan atau data peserta workshop.

3. Sistem Mengecek Stok/Kuota:

- Jika stok/kuota mencukupi, sistem mengizinkan invoice diterbitkan dan stok/kuota otomatis berkurang (atau berkurang saat status berubah jadi DP/Lunas).

4. Pilih Opsi Pengiriman Invoice:

-  **Kirim WhatsApp:** Mengirimkan pesan berisi link invoice dengan *badge* status warna terkait.
-  **Kirim Email:** Email masuk dengan lampiran PDF.
-  **Print:** Cetak dokumen dengan stempel indikator warna **LUNAS (Hijau)**, **DP (Kuning)**, atau **BELUM LUNAS (Merah)**.

4. Keuntungan Struktur Ini untuk DynoBoo

1. **Pengelolaan Stok Fleksibel:** Stok produk fisik dan kuota workshop dikelola dalam satu modul master barang yang terintegrasi.
2. **Laporan Penjualan Terpisah:** Anda bisa dengan mudah melihat omzet bulanan: berapa hasil dari **penjualan produk rajutan** dan berapa hasil dari **penyelenggaraan workshop**.
3. **Pemberitahuan Status Pembayaran Jelas:** Kategori warna (*Hijau, Kuning, Merah*) meminimalisir salah paham dengan pembeli mengenai sisa tagihan DP atau pelunasan.

User prompt: A. Tabel item_types (Jenis Produk & Event) minta di perjelas, jadi ini database fungsi dalam membantu pas pencetakan invoice nanti, jadi mikrinya user bisa buat kode item-types tersendiri dan ada catatan tersendiri (untuk nanti di invoice di cetak, soalnya setiap invocie nmemiliki catatan yang berbeda - beda) Tabel items udh oke, tapi tabel stock aku mikir buat terpisah, jadi intinya buat konsepnya macam POS point of sales tapi secara online di satu web admin gitu soalnya konsepnya bukan untuk kasir offline taspi untuk bisnis online gfitu

Response: Paham sekali! Konsep **Online Admin POS (Point of Sales)** ini sangat cocok untuk operasional bisnis online DynoBoo.

Admin tidak menggunakan mesin kasir fisik, melainkan sebuah **Dashboard Admin Web** yang berfungsi sebagai "meja kasir & pusat kendali online". Dari satu halaman ini, Admin bisa input pesanan masuk, memotong stok otomatis, menerbitkan invoice lengkap dengan catatan khusus per jenis produk, lalu langsung mengirimkannya via WhatsApp, Email, atau Cetak PDF.

Berikut adalah pembaruan rancangan konsep dan database yang disesuaikan dengan kebutuhan tersebut.

1. Pembaruan Tabel item_types (Jenis Produk & Event)

Fungsi utama tabel ini sekarang memiliki 2 peranan penting:

1. **Pengelompokan & Kode Kategori** untuk memudahkan pencarian di POS Admin.
2. **Template Catatan/Syarat & Ketentuan Otomatis** pada saat pencetakan Invoice.

Structure Tabel item_types

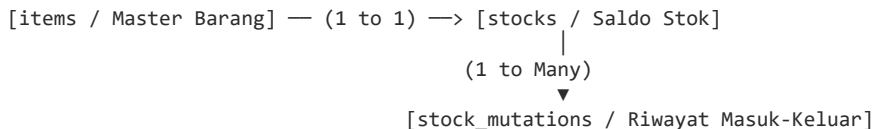
Field Name	Type	Description	Contoh Isi Data
id (PK)	INT / UUID	ID Unik Jenis	1
type_code	VARCHAR(20)	Kode Unik Jenis (<i>Kustom User</i>)	WS-OFF / RJT-PO / RJT-READY
name	VARCHAR(100)	Nama Jenis	"Workshop Offline", "Rajut Custom PO"
default_invoice_notes	TEXT	Catatan/Syarat bawaan yang akan muncul di cetakan invoice	"Syarat Workshop: Harap hadir 15 menit sebelum sesi. Tiket yang sudah dibeli tidak dapat di-refund."

Bagaimana fitur ini bekerja saat buat Invoice di Admin POS?

- Saat Admin memilih barang jenis "**Workshop Offline**", sistem akan otomatis mengambil *text* dari `default_invoice_notes` jenis tersebut dan mengisikannya ke kolom catatan invoice.
- Admin masih **bisa mengedit/menambah catatan tersebut secara fleksibel** khusus untuk invoice yang sedang dibuat sebelum dicetak.

2. Pemisahan Database Stok (Tabel stocks Terpisah)

Memisahkan tabel stok dari tabel master item adalah praktik terbaik untuk sistem POS. Hal ini membuat pencatatan alur masuk-keluar barang (mutasi stok) menjadi transparan dan akurat.



A. Tabel stocks (Saldo Stok Real-time)

Tabel ini mencatat sisa stok atau kuota saat ini.

Field Name	Type	Description	Contoh Isi
id (PK)	INT / UUID	ID Stok	1
item_id (FK)	INT / UUID	Relasi ke items.id	101 (Amigurumi Dino)
qty_available	INT	Sisa Stok / Sisa Kuota Kursi	15
min_stock_alert	INT	Batas minimal untuk notifikasi stok habis	2

B. Tabel stock_mutations (Riwayat/Log Keluar-Masuk Stok)

Mencatat *siapa*, *kapan*, dan *alasan* stok bertambah atau berkurang (sangat berguna untuk audit POS).

Field Name	Type	Description	Contoh Isi Data
id (PK)	INT / UUID	ID Mutasi	1
item_id (FK)	INT / UUID	Relasi ke items.id	101
invoice_id (FK)	INT / UUID	Relasi ke invoice (nullable)	INV/DNB/2026/001
type	ENUM	Jenis mutasi	IN (Tambah Stok), OUT (Terjual via Invoice), ADJUSTMENT (Penyesuaian)
qty	INT	Jumlah perubahan (+/-)	-2 (berkurang 2 pcs)
notes	TEXT	Keterangan mutasi	"Penjualan via Invoice #INV-001" atau "Restock benang baru"

3. Konsep Alur Kerja "Online POS Admin" DynoBoo

[Dashboard POS Admin]

- 1. Pilih / Tambah Pelanggan
- 2. Pilih Item (Sistem cek 'stocks', otomatis ambil 'type_code' & 'default_invoice_notes')
- 3. Input Custom Notes per Item (misal: "Warna Benang: Pink Pastel")
- 4. Input Nominal Pembayaran (Sistem kalkulasi Sisa & Tentukan Status: Hijau/Kuning/Merah)
- [EKSEKUSI TRASAKSI]
 - Stok Otomatis Berkurang (Tercatat di 'stock_mutations')
 - Kirim Invoice (WhatsApp / Email / Print PDF)

Keunggulan Alur Online POS Ini:

- Pencatatan Otomatis & Cepat:** Admin cukup klik-klik item layaknya di kasir, tapi disesuaikan untuk pesanan jarak jauh/online.
- Indikator Pembayaran Jelas:**
 - **Lunas (Hijau):** Langsung siap masuk antrean kirim / pengerjaan.
 - **DP (Kuning):** Masuk antrean produksi (PO) atau reservasi kuota workshop.
 - **Belum Lunas (Merah):** Menunggu pembayaran awal.
- Fleksibilitas Cetak Invoice:** Saat invoice dicetak (PDF/Print), layout cetakan memuat:
 - Header Toko (DynoBoo).
 - Status Badge Warna.
 - List Barang + Detail Custom Note Item.
 - **Catatan Khusus Jenis (dari item_types.default_invoice_notes)** di bagian footer invoice (contoh: aturan PO, ketentuan workshop, atau nomor rekening pelunasan).

Apakah struktur pemisahan stok dan template catatan per jenis barang ini sudah sesuai dengan ekspektasi operasional DynoBoo, atau ada kebutuhan pencatatan lain yang ingin ditambahkan?

User prompt: Untuk database-nya coba diperjelas ya. Jadi ini konsepnya ini aku mau bikin website untuk handicrafter. Jadi kurang lebih untuk bahan-bahan untuk produk-produk rajutan serta event-event workshop misalnya pelatihan rajut, rajut pot bunga atau apalah gitu. Nah, yang menjadi pertimbangan aku adalah website-nya ini aku mau menormalisasi ya jadi lebih kompleks agak kompleks ya. Jadi ada nama barang, tadi seperti eh bukan ada nama produk, oke nama produk terus kita lari ke invoice. Invoice-nya ini aku pengen terpisah soalnya setiap invoice dalam kutip kalau misalnya orang beli barang sama ngikuti workshop kan invoice-nya berbeda ya syarat, ketentuan bahkan kode invoice-nya. Nah, makanya aku mau bikin satu database terpisah di mana user admin bisa mengelola invoice mereka sesuka hati. Dan terus ada satu lagi artinya database-nya terpisah dong. Satu satu untuk jenis-jenisnya, satu untuk riwayat transaksinya. Jadi satu database itu untuk riwayat transaksi. Nah, terus untuk stok barang. Stok barang ini juga pisah. Jadi stok barang ini eh ya stok barang enggak usah terpisahlah. Stok barang satu database di mana tetapnya ada di dalamnya ada quantity quantity yang tersedia sama quantity terjual. Quantity terjual tuh di mana? Dekat nanti yang koneksi ke relasi ke history database history penjualan ini. Nah, oke nama barang udah oke, history penjualan udah oke. Nama barang sama jenis produk bisa ya database-nya. Jadi ada nama barang, nama barang maksudnya produknya apa. Soalnya gini, kalau jenis barangnya misalnya ada jenis barang misalnya diatur produk jenis barang produk jadi produk jadi. Nah, kan produk jadi itu bisa banyak ya barangnya apa gitu. Ya anggap SKU gitu lah tapi bukan SKU dalam kutip beli produk barang ini cuman dia konteksnya ya handicrafter gitu. Terus point of sale history laporan. Jadi aku juga mau minta dibikinkan database bukan database ini untuk front end sih. Nanti fokus database dulu.

Response: Pendekatan normalisasi database ini sangat tepat. Memisahkan antara **Konfigurasi Jenis Invoice, Katalog Produk & Event, Stok**, serta **Riwayat Transaksi** akan membuat sistem DynoBoo sangat modular, rapi, dan mudah dikembangkan di masa depan.

Berikut adalah rancangan struktur database ter-normalisasi yang dirancang khusus sesuai dengan kebutuhan transaksi produk handicraft dan event workshop.

1. Kelompok Katalog (Jenis & Master Item)

Memisahkan kategori ("Jenis") dan item spesifiknya agar admin fleksibel menambah jenis produk baru (misal: Produk Jadi, Bahan/Kit Rajut, Workshop Offline, Workshop Online).

A. Tabel item_types (Jenis Produk / Event)

Mengatur pengelompokan produk dan sifat dasarnya.

Field Name	Type	Description	Contoh Isi
id (PK)	INT / UUID	ID Unik Jenis	1
type_code	VARCHAR(20)	Kode Kategori (<i>Kustom</i>)	PRD-JDI, BHN-RJT, WKS-OFF
type_name	VARCHAR(100)	Nama Jenis	"Produk Jadi", "Bahan Rajut", "Workshop Offline"
is_event	BOOLEAN	Penanda apakah ini workshop/event (true) atau fisik (false)	false / true

B. Tabel items (Master Produk & Event)

Menyimpan detail spesifik setiap barang atau event yang dijual.

Field Name	Type	Description	Contoh Isi
id (PK)	INT / UUID	ID Unik Item	101
item_type_id (FK)	INT / UUID	Relasi ke item_types.id	1
item_code	VARCHAR(50)	Kode Unik Item / SKU Handicraft	AMIG-DNO-01, WS-POT-BGA
name	VARCHAR(150)	Nama Produk / Nama Workshop	"Amigurumi Dino Pink", "Workshop Rajut Pot Bunga"
price	DECIMAL(12,2)	Harga Satuan / Biaya Tiket	150000.00
description	TEXT	Deskripsi singkat produk/event	"Termasuk benang dan kail rajut..."
is_active	BOOLEAN	Status aktif/dijual	true

2. Kelompok Stok Barang & Kuota (stocks)

Satu tabel khusus untuk mengelola sisa stok tersedia dan jumlah yang sudah terjual. Tabel ini terhubung 1-to-1 dengan tabel items.

Tabel stocks

Field Name	Type	Description	Contoh Isi
id (PK)	INT / UUID	ID Unik Stok	1
item_id (FK)	INT / UUID	Relasi ke items.id (Unique)	101
qty_available	INT	Jumlah stok/kuota yang tersedia	15

Field Name	Type	Description	Contoh Isi
qty_sold	INT	Jumlah akumulasi yang terjual	35
min_stock_alert	INT	Batas notifikasi stok menipis	3

Logika Relasi Stok: Ketika transaksi terjadi di `invoice_items` dengan status terkonfirmasi (DP/Lunas):

- `qty_available` otomatis berkurang.
- `qty_sold` otomatis bertambah sesuai jumlah yang dibeli.

3. Kelompok Pengaturan Invoice Terpisah (`invoice_settings`)

Tabel ini mawadahi permintaan Anda agar Admin dapat mengelola **format kode invoice, syarat, ketentuan, dan aturan khusus** secara bebas antara transaksi barang fisik dan event workshop.

Tabel `invoice_settings` (Template & Rules Invoice)

Field Name	Type	Description	Contoh Isi
<code>id</code> (PK)	INT / UUID	ID Unik Pengaturan	1
<code>setting_name</code>	VARCHAR(100)	Nama Pengaturan/Kategori	"Invoice Produk Handicraft", "Invoice Workshop"
<code>code_prefix</code>	VARCHAR(20)	Prefix Kode Invoice	INV-DNB-PRD, INV-DNB-WKS
<code>terms_and_conditions</code>	TEXT	Syarat & Ketentuan yang tercetak di invoice	"Garansi retur 2 hari untuk barang cacat..." ATAU "Tiket workshop tidak dapat dibatalkan..."
<code>footer_note</code>	TEXT	Catatan Tambahan (Misal No. Rekening)	"Transfer BCA: 12345678 a.n DynoBoo"

4. Kelompok Riwayat Transaksi (History Database)

Modul ini khusus menyimpan seluruh jejak transaksi yang pernah terjadi secara permanen.

A. Tabel `customers` (Data Pelanggan)

Field Name	Type	Description
<code>id</code> (PK)	INT / UUID	ID Pelanggan
<code>name</code>	VARCHAR(100)	Nama Pelanggan
<code>phone</code>	VARCHAR(20)	Nomor WhatsApp
<code>email</code>	VARCHAR(100)	Email
<code>address</code>	TEXT	Alamat Pengiriman

B. Tabel `invoices` (Header Riwayat Penjualan)

Mencatat header transaksi, status pembayaran, dan mengunci aturan invoice yang dipakai saat transaksi dibuat.

Field Name	Type	Description	Contoh Isi
<code>id</code> (PK)	INT / UUID	ID Unik Invoice	1001
<code>invoice_number</code>	VARCHAR(50)	Nomor Invoice Resmi	INV-DNB-WKS/202607/005
<code>invoice_setting_id</code> (FK)	INT / UUID	Relasi ke <code>invoice_settings.id</code>	2 (Pengaturan Workshop)
<code>customer_id</code> (FK)	INT / UUID	Relasi ke <code>customers.id</code>	50
<code>issue_date</code>	DATE	Tanggal Dibuat	2026-07-29
<code>due_date</code>	DATE	Tanggal Jatuh Tempo	2026-08-01
<code>subtotal</code>	DECIMAL(12,2)	Total harga barang/tiket	300000.00
<code>shipping_cost</code>	DECIMAL(12,2)	Ongkos Kirim (0 jika workshop)	15000.00
<code>discount</code>	DECIMAL(12,2)	Potongan Harga	0.00

Field Name	Type	Description	Contoh Isi
total_amount	DECIMAL(12,2)	Total Akhir Tagihan	315000.00
amount_paid	DECIMAL(12,2)	Total Uang Masuk	100000.00
payment_status	ENUM	Status Pembayaran	'LUNAS' (Hijau), 'DP' (Kuning), 'BELUM_LUNAS' (Merah)
applied_terms	TEXT	Snapshot Syarat & Ketentuan saat invoice ini diterbitkan	"Tiket workshop tidak dapat dibatalkan..."

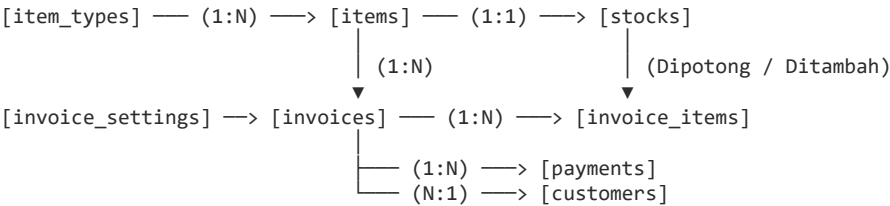
C. Tabel invoice_items (Detail Rincian Barang/Event Terjual)

Field Name	Type	Description	Contoh Isi
id (PK)	INT / UUID	ID Unik Item Transaksi	5001
invoice_id (FK)	INT / UUID	Relasi ke invoices.id	1001
item_id (FK)	INT / UUID	Relasi ke items.id	101
item_name_snapshot	VARCHAR(150)	Nama item saat transaksi	"Workshop Rajut Pot Bunga"
qty	INT	Jumlah yang dibeli	2
price	DECIMAL(12,2)	Harga per unit saat transaksi	150000.00
custom_notes	TEXT	Catatan kustomisasi	"Peserta 1: Budi, Peserta 2: Ani"
subtotal	DECIMAL(12,2)	(qty * price)	300000.00

D. Tabel payments (Riwayat Pembayaran DP & Pelunasan)

Field Name	Type	Description	Contoh Isi
id (PK)	INT / UUID	ID Pembayaran	1
invoice_id (FK)	INT / UUID	Relasi ke invoices.id	1001
payment_date	DATETIME	Tanggal & Waktu Bayar	2026-07-29 14:00:00
amount	DECIMAL(12,2)	Nominal yang dibayar	100000.00 (DP 1)
payment_method	VARCHAR(50)	Metode Pembayaran	"Transfer BCA"
proof_file	VARCHAR(255)	Path Bukti Transfer	"proofs/inv-1001-dp1.jpg"

Visualisasi Ringkas Diagram Relasi Database (ERD)



Keunggulan Skema Ini:

- 1. **Pemisahan Logika Invoice:** Admin bisa mengubah template syarat/ketentuan workshop tanpa memengaruhi syarat & ketentuan produk handicraft.
- 2. **Historis Terkunci:** Penggunaan *field snapshot* (item_name_snapshot, applied_terms) menjaga agar riwayat transaksi lama tidak ikut berubah apabila di masa depan master produk atau template invoice diedit oleh admin.
- 3. **Pencatatan Stok Akurat:** Pergerakan qty_available dan qty_sold terikat langsung secara otomatis dengan entri pada invoice_items.

User prompt: itu contoh invoice yang diharapkan nanti versi digital PDF, Nah, dari gambar ini artinya kan perlu lagi ya satu buat nama penjual. Artinya dua tabel terpisah dong. Satu tabelnya itu berisi identitas setiap mereka beli. Soalnya kan user bisa beli tiga barang, empat barang, ya dong. Artinya itu yang satu database-nya buat mengelola itu semua. Nah, satunya lagi transaksi ini buat cetak invoice. Jadi ada dua database. Satu yang buat fokus ke invoice, satunya buat isi apa yang user beli dekat invoice-nya. Terpisah dong.

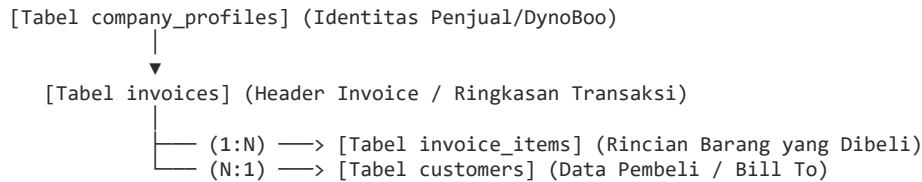
Response: Logika dan pemikiran kamu **100% tepat!** Dalam perancangan database (skema relasional), konsep ini dinamakan relasi **Parent-Child (One-to-Many / 1:N)**.

Satu transaksi/invoice (*Parent*) bisa menampung 1, 3, 10, atau banyak barang sekaligus (*Child*). Jika ditaruh dalam satu tabel saja, datanya akan berantakan dan terjadi *redundancy* (pengulangan data yang tidak perlu).

Selain itu, ide menambahkan **Tabel Identitas Penjual (Profil Toko DynoBoo)** juga sangat bagus agar logo, nama toko, alamat, dan kontak toko tidak di-*hardcode* di dalam kodingan website.

Pemetaan Elemen Gambar ke Struktur Database

Jika kita bedah gambar acuan invoice kamu, berikut adalah pemetaannya ke dalam tabel database terpisah:



Detail Rancangan Tabel Database

1. Tabel Identitas Penjual (company_profiles)

Menyimpan data profil DynoBoo yang muncul di bagian header invoice (Logo, Subtitle, Kontak).

Field Name	Type	Keterangan & Contoh Isi Gambar
id (PK)	INT / UUID	ID Profil
company_name	VARCHAR(100)	"DynoBoo"
tagline	VARCHAR(150)	"Handmade Crochet Dolls & Beaded Accessories"
logo_url	VARCHAR(255)	Path file logo DynoBoo
phone_number	VARCHAR(20)	Nomor WhatsApp Toko
address	TEXT	Alamat Toko/Studio DynoBoo

2. Tabel Header Invoice (invoices) — *Tabel Utama 1*

Fokus menyimpan **ringkasan transaksi, tanggal, nomor invoice, total harga, dan catatan**.

Field Name	Type	Keterangan & Contoh Isi Gambar
id (PK)	INT / UUID	ID Unik Transaksi
invoice_number	VARCHAR(50)	DNB-WSP-2608-0003
invoice_type_name	VARCHAR(100)	Invoice Bouquet / Invoice Workshop
customer_name	VARCHAR(100)	Bill To : Nama Konsumen
issue_date	DATE	28 Juli 2026
payment_status	ENUM	'LUNAS' (Hijau), 'DP' (Kuning), 'BELUM_LUNAS' (Merah)
subtotal	DECIMAL(12,2)	Total harga barang (sebelum diskon): Rp 100.000
discount	DECIMAL(12,2)	Total Diskon: Rp 0
dp_amount	DECIMAL(12,2)	Total DP yang sudah masuk: Rp 0
grand_total	DECIMAL(12,2)	Tagihan Akhir (subtotal - discount): Rp 100.000
terms_and_notes	TEXT	Catatan/T&C di bagian bawah invoice

3. Tabel Detail Rincian Barang (invoice_items) — *Tabel Utama 2*

Fokus mengelola **apa saja barang/workshop yang dibeli** dalam invoice tersebut. Tabel ini bisa berisi **banyak baris** untuk 1 transaksi invoice.

Field Name	Type	Keterangan & Contoh Isi Gambar
id (PK)	INT / UUID	ID Item Transaksi

Field Name	Type	Keterangan & Contoh Isi Gambar
invoice_id (FK)	INT / UUID	Kunci Penghubung ke invoices.id
description	VARCHAR(255)	Description: "Workshop Animal Pot"
qty	INT	QTY.: 1 (atau 3, 4 jika beli banyak)
unit_name	VARCHAR(20)	Satuan: "Pcs", "Tiket", "Set"
price	DECIMAL(12,2)	Price: Rp 100.000
total_price	DECIMAL(12,2)	Total (qty * price): Rp 100.000

Contoh Simulasi Data Jika User Beli 3 Barang Sekaligus

Misalnya pelanggan bernama **Siti** membeli 3 jenis produk dalam 1 transaksi:

A. Entri di Tabel invoices (Cukup 1 Baris Data):

- id: 101
- invoice_number: DNB-PRD-2607-0012
- customer_name: Siti
- grand_total: Rp 350.000

B. Entri di Tabel invoice_items (Ada 3 Baris Data Menunjuk ke invoice_id = 101):

id	invoice_id	description	qty	price	total_price
1	101	Amigurumi Dino Pink	2 Pcs	Rp 100.000	Rp 200.000
2	101	Gantungan Kunci Rajut	1 Pcs	Rp 50.000	Rp 50.000
3	101	Workshop Rajut Pot Bunga	1 Tiket	Rp 100.000	Rp 100.000

Bagaimana Sistem Cetak PDF Membacanya? Saat cetak invoice PDF/Print dijalankan, sistem web akan mengambil **1 data dari invoices** untuk bagian Atas (Header & Bill To) dan Bawah (Grand Total & Catatan), lalu mengambil **semua baris dari invoice_items yang invoice_id = 101** untuk di-looping menjadi baris-baris tabel produk.

Pemisahan skema ini sudah sangat aman, sesuai standar industri *Point of Sales* (POS), dan memudahkan pencetakan PDF yang dinamis!

User prompt: Nah, untuk invoice number-nya ini mereka ini mau otomatis. Makanya tadi aku minta kau bikin kode tersendiri kan. Jadi, invoice number-nya itu bukan dari bukan dari kamu, bukan dari saya maksudnya, bukan dari orang yang ketik tetapi dari generate yang pas setiap kali mereka transaksi dari tipe tipe tipe inilah tipe transaksi lah. Artinya perlu ada tipe invoice. Ya, kan ada database invoice tadi kan yang aku minta ada tambah deskripsi itu kan. Yang minta deskripsi terus ini kan. Iya. Iya, ya.

Response: Sangat paham! Penomoran otomatis (*auto-generated invoice number*) berdasarkan Tipe Transaksi adalah standar terbaik dalam sistem POS/ERP.

Dengan cara ini, Admin **tidak perlu mengetik nomor invoice secara manual** (mencegah salah ketik atau nomor ganda), dan setiap jenis transaksi akan memiliki format penomoran serta catatan/syarat & ketentuan yang sesuai secara otomatis.

Berikut adalah penyempurnaan konsep dan struktur database untuk menangani **Tipe Invoice & Generator Nomor Otomatis**:

1. Tabel Master Tipe Invoice (invoice_types)

Tabel ini dikelola oleh Admin untuk menentukan jenis-jenis transaksi yang ada di DynoBoo beserta aturan penomorannya.

Field Name	Type	Description	Contoh Isi Data
id (PK)	INT / UUID	ID Unik Tipe	1
type_code	VARCHAR(10)	Kode Singkatan Tipe	WSP (Workshop) / BQT (Bouquet) / RJT (Rajut/Handicraft)
type_name	VARCHAR(100)	Nama Tipe Transaksi	"Invoice Workshop", "Invoice Bouquet"
code_prefix	VARCHAR(20)	Awalan Kode Invoice	DNB-WSP, DNB-BQT, DNB-RJT
default_terms	TEXT	Catatan/S&K Otomatis	"Dengan melakukan pembayaran, peserta dianggap telah membaca dan menyetujui..."

Field Name	Type	Description	Contoh Isi Data
is_active	BOOLEAN	Status Tipe Aktif/Tidak	true

2. Tabel Counter Penomoran (invoice_counters)

Agar nomor urut tidak pernah bentrok dan bisa reset otomatis (misalnya berganti bulan/tahun), kita buatkan tabel pembantu *Counter*.

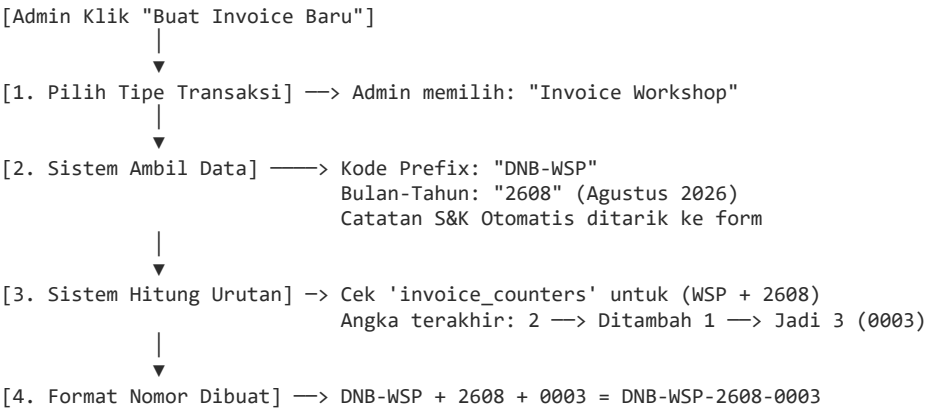
Field Name	Type	Description	Contoh Isi Data
id (PK)	INT / UUID	ID Counter	1
invoice_type_id (FK)	INT / UUID	Relasi ke invoice_types.id	1 (Tipe Workshop)
year_month	VARCHAR(6)	Tahun dan Bulan (Format: YYMM)	2608 (Tahun 2026, Bulan 08)
last_sequence	INT	Nomor urut terakhir yang dipakai	3 (artinya invoice berikutnya adalah 4)

3. Tabel Header Invoice (invoices)

Tabel ini menyimpan hasil *generate* nomor invoice dan relasinya ke Tipe Invoice.

Field Name	Type	Description	Contoh Isi Ter-generate
id (PK)	INT / UUID	ID Unik Transaksi	1001
invoice_type_id (FK)	INT / UUID	Relasi ke invoice_types.id	1
invoice_number	VARCHAR(50)	Nomor Otomatis dari System	DNB-WSP-2608-0003
customer_name	VARCHAR(100)	Nama Konsumen	"Nama Konsumen"
issue_date	DATE	Tanggal Buat	2026-07-28
payment_status	ENUM	Status Pembayaran	'LUNAS' / 'DP' / 'BELUM_LUNAS'
grand_total	DECIMAL(12,2)	Total Akhir	100000.00
terms_and_notes	TEXT	Catatan (Auto dari default_terms)	"Dengan melakukan pembayaran..."

Cara Kerja Sistem Saat Admin Membuat Invoice Baru



Contoh Hasil Penomoran Berdasarkan Tipe:

- 1. Skenario 1 (Transaksi Workshop ke-3 di Bulan Agustus 2026):
 - Tipe yang dipilih: *Invoice Workshop*
 - Nomor Otomatis: DNB-WSP-2608-0003
- 2. Skenario 2 (Transaksi Produk Bouquet ke-1 di Bulan Agustus 2026):
 - Tipe yang dipilih: *Invoice Bouquet*
 - Nomor Otomatis: DNB-BQT-2608-0001
- 3. Skenario 3 (Transaksi Rajutan/Handicraft ke-12 di Bulan Agustus 2026):

- Tipe yang dipilih: *Invoice Rajut*
- Nomor Otomatis: *DNB-RJT-2608-0012*

Keuntungan Sistem Ini Bagi DynoBoo:

- 1. **Praktis & Cepat:** Admin cukup pilih Tipe Transaksi via *dropdown*, lalu nomor invoice dan syarat ketentuan langsung terisi otomatis.
- 2. **Laporan Rapi:** Di database, Kamu bisa dengan mudah memfilter: *"Berapa total omzet dari DNB-WSP (Workshop) bulan ini dibanding DNB-BQT (Bouquet)?"*.
- 3. **Fleksibel:** Jika bulan depan DynoBoo menambah jenis transaksi baru (misalnya *Invoice Bahan/Kit Rajut* dengan kode DNB-KIT), Admin cukup menambahkannya di tabel *invoice_types* tanpa perlu mengubah kodingan website.

User prompt: *coba tulis ulang semua databasenya dalam pdf berserta sqlnya*

Response: Sepertinya Anda belum melampirkan skema, data, atau struktur database yang ingin ditulis ulang.

Sebagai catatan, sebagai AI saya tidak bisa mengunduh atau mengirimkan file fisik berkesteni .pdf secara langsung ke perangkat Anda. Namun, saya bisa membantu Anda dengan **dua alternatif berikut**:

- 1. **Format Markdown Rapi:** Saya akan menuliskan seluruh skema database, relasi, data, dan query SQL lengkap dengan struktur visual yang rapi. Anda bisa langsung menyalinnya ke text editor (seperti VS Code, Obsidian, atau Google Docs) lalu memilih menu **Print / Save as PDF**.
- 2. **Script Generator PDF (Python):** Saya bisa membuatkan script Python (menggunakan library *fpdf2* atau *reportlab*) yang dapat Anda jalankan di komputer/Jupyter Notebook untuk secara otomatis mengeksport seluruh skema dan kode SQL menjadi file PDF yang tertata bersih.

Contoh Format Dokumentasi SQL yang Bisa Dibuat

```
-- =====
-- SKEMA DATABASE: Manajemen Pengguna
-- =====

CREATE TABLE users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(50) NOT NULL UNIQUE,
  email VARCHAR(100) NOT NULL UNIQUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Contoh Input Data
INSERT INTO users (username, email)
VALUES ('budi_santoso', 'budi@example.com');
```

Bisa tolong bagikan detail struktur tabel, nama field, atau topik database yang ingin Anda buatkan SQL dan dokumentasinya?

User prompt: *coba tuliskan ulang semua databasenya*

Response: Berikut adalah rekapitulasi lengkap seluruh **Rancangan Database, Kamus Data, dan Script SQL DDL/DML** yang ter-normalisasi untuk website **DynoBoo (Handicraft & Workshop)**.

1. Ringkasan Arsitektur & Relasi Tabel (ERD Overview)

Database ini terdiri dari **10 tabel terpisah** yang terbagi dalam 5 kelompok modul utama:

```
[Modul Profil Toko]
└─ company_profiles — (Dipanggil saat cetak Header Invoice)

[Modul Auto Numbering & Rule Invoice]
├─ invoice_types — (Mengatur Prefix: WSP, BQT, RJT & Syarat Ketentuan)
└─ invoice_counters — (Menghitung urutan transaksi bulanan: 2608-0003)

[Modul Katalog & Stok]
├─ item_types — (Kategori: Produk Rajut vs Event Workshop)
├─ items — (Master Katalog Barang & Tiket)
└─ stocks — (Kelola qty_available & qty_sold secara terpisah)

[Modul Transaksi Invoice (Parent-Child)]
├─ customers — (Data Pembeli / Bill To)
├─ invoices — (Header Transaksi, Status Warna, Total Tagihan)
└─ invoice_items — (Rincian 1, 3, atau banyak barang dalam 1 invoice)

[Modul Pembayaran]
└─ payments — (Riwayat Angsuran DP & Pelunasan)
```

2. Kamus Data & Struktur Tabel

A. Identitas Penjual & Konfigurasi Invoice

1. Tabel `company_profiles` (Profil Penjual / DynoBoo)

Fungsi: Menyimpan identitas toko agar logo, nama, dan alamat tidak di-hardcode di kodingan web.

- `id` (INT, PK, AUTO_INCREMENT)
- `company_name` (VARCHAR(100)) — Contoh: "DynoBoo"
- `tagline` (VARCHAR(150)) — Contoh: "Handmade Crochet Dolls & Beaded Accessories"
- `logo_path` (VARCHAR(255)) — Path file logo DynoBoo
- `phone_number` (VARCHAR(20)) — Nomor Kontak WhatsApp Toko
- `email` (VARCHAR(100))
- `address` (TEXT)

2. Tabel `invoice_types` (Master Tipe Invoice)

Fungsi: Mengatur kode prefix dan template Syarat & Ketentuan (S&K) otomatis per jenis transaksi.

- `id` (INT, PK, AUTO_INCREMENT)
- `type_code` (VARCHAR(10), UNIQUE) — Contoh: 'WSP', 'BQT', 'RJT'
- `type_name` (VARCHAR(100)) — Contoh: "Invoice Workshop", "Invoice Bouquet"
- `code_prefix` (VARCHAR(20)) — Contoh: "DNB-WSP", "DNB-BQT", "DNB-RJT"
- `default_terms` (TEXT) — Template S&K otomatis yang tampil di footer invoice
- `is_active` (BOOLEAN) — Default: TRUE

3. Tabel `invoice_counters` (Penomoran Otomatis)

Fungsi: Menyimpan urutan terakhir penomoran invoice per tipe dan per periode bulan/tahun.

- `id` (INT, PK, AUTO_INCREMENT)
- `invoice_type_id` (INT, FK -> `invoice_types.id`)
- `year_month` (VARCHAR(6)) — Format YYMM (Contoh: '2608' untuk Agustus 2026)
- `last_sequence` (INT) — Urutan terakhir (Contoh: 3 untuk DNB-WSP-2608-0003)

B. Katalog Produk & Pengelolaan Stok

4. Tabel `item_types` (Jenis Produk & Event)

Fungsi: Membedakan antara produk fisik (rajutan) dan jasa event (workshop).

- `id` (INT, PK, AUTO_INCREMENT)
- `category_code` (VARCHAR(20), UNIQUE) — Contoh: 'PRD-READY', 'PRD-PO', 'WKS-OFF'
- `category_name` (VARCHAR(100)) — Contoh: "Produk Ready Stock", "Workshop Offline"
- `is_event` (BOOLEAN) — TRUE untuk Event/Workshop, FALSE untuk Produk Fisik

5. Tabel `items` (Master Produk & Workshop)

Fungsi: Katalog seluruh item yang dijual DynoBoo.

- `id` (INT, PK, AUTO_INCREMENT)
- `item_type_id` (INT, FK -> `item_types.id`)
- `item_code` (VARCHAR(50), UNIQUE) — Contoh: 'WKS-POT-01', 'AMIG-DNO-01'
- `name` (VARCHAR(150)) — Contoh: "Workshop Animal Pot", "Amigurumi Dino Pink"
- `unit_name` (VARCHAR(20)) — Contoh: 'Pcs', 'Tiket', 'Set'
- `price` (DECIMAL(12,2)) — Harga Satuan / Biaya Pendaftaran
- `description` (TEXT)

- `is_active` (BOOLEAN) — Default: TRUE

6. Tabel stocks (Stok & Kuota Terpisah)

Fungsi: Mengelola jumlah barang/kuota yang tersedia dan akumulasi yang sudah terjual.

- `id` (INT, PK, AUTO_INCREMENT)
- `item_id` (INT, FK -> `items.id`, UNIQUE 1:1)
- `qty_available` (INT) — Jumlah stok/kuota yang **tersedia saat ini**
- `qty_sold` (INT) — Akumulasi jumlah **terjual**
- `min_stock_alert` (INT) — Batas peringatan stok menipis

C. Transaksi Penjualan & Pembayaran

7. Tabel customers (Data Pembeli)

- `id` (INT, PK, AUTO_INCREMENT)
- `name` (VARCHAR(100)) — Nama Konsumen
- `phone` (VARCHAR(20)) — Nomor WhatsApp
- `email` (VARCHAR(100))
- `address` (TEXT) — Alamat Pengiriman

8. Tabel invoices (Header Transaksi Utama)

Fungsi: Ringkasan transaksi, status pembayaran, dan penomoran otomatis.

- `id` (INT, PK, AUTO_INCREMENT)
- `invoice_number` (VARCHAR(50), UNIQUE) — **Auto Generated** (misal: DNB-WSP-2608-0003)
- `invoice_type_id` (INT, FK -> `invoice_types.id`)
- `customer_id` (INT, FK -> `customers.id`)
- `issue_date` (DATE) — Tanggal Penerbitan (misal: 2026-07-28)
- `due_date` (DATE) — Tanggal Jatuh Tempo
- `subtotal` (DECIMAL(12,2)) — Subtotal Tagihan
- `discount` (DECIMAL(12,2)) — Diskon
- `dp_amount` (DECIMAL(12,2)) — Nominal DP Terbayar
- `grand_total` (DECIMAL(12,2)) — Total Akhir Tagihan (`subtotal - discount`)
- `payment_status` (ENUM) — 'LUNAS' (● **Hijau**), 'DP' (● **Kuning**), 'BELUM_LUNAS' (● **Merah**)
- `terms_and_notes` (TEXT) — Snapshot S&K saat invoice dicetak

9. Tabel invoice_items (Rincian Barang Terjual)

Fungsi: Menyimpan rincian item yang dibeli (bisa menampung banyak item untuk 1 invoice).

- `id` (INT, PK, AUTO_INCREMENT)
- `invoice_id` (INT, FK -> `invoices.id`)
- `item_id` (INT, FK -> `items.id`, NULLable)
- `description` (VARCHAR(255)) — Deskripsi barang (misal: "Workshop Animal Pot")
- `qty` (INT) — Jumlah dibeli
- `unit_name` (VARCHAR(20)) — Satuan (misal: 'Pcs', 'Tiket')
- `price` (DECIMAL(12,2)) — Harga per unit saat transaksi
- `total_price` (DECIMAL(12,2)) — Hasil kalkulasi (`qty * price`)
- `custom_notes` (TEXT) — Catatan kustom (misal: warna benang / nama peserta)

10. Tabel payments (Riwayat Pembayaran)

- `id` (INT, PK, AUTO_INCREMENT)

- invoice_id (INT, FK -> invoices.id)
- payment_date (DATETIME) — Waktu bayar diterima
- amount_paid (DECIMAL(12,2)) — Nominal uang masuk
- payment_method (VARCHAR(50)) — Contoh: "Transfer BCA", "QRIS", "Cash"
- proof_file (VARCHAR(255)) — Path foto bukti transfer
- notes (VARCHAR(255))

3. Script SQL DDL (Create Table, Relasi, & Index)

Anda dapat menyalin dan mengeksekusi script SQL ini langsung pada MySQL/MariaDB/PostgreSQL:

```
-- =====
-- DATABASE CREATION & SELECTION
-- =====
CREATE DATABASE IF NOT EXISTS dynoboo_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE dynoboo_db;

-- 1. TABEL PROFIL TOKO (company_profiles)
CREATE TABLE IF NOT EXISTS company_profiles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  company_name VARCHAR(100) NOT NULL,
  tagline VARCHAR(150) NULL,
  logo_path VARCHAR(255) NULL,
  phone_number VARCHAR(20) NOT NULL,
  email VARCHAR(100) NULL,
  address TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 2. TABEL TIPE INVOICE & S&K (invoice_types)
CREATE TABLE IF NOT EXISTS invoice_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  type_code VARCHAR(10) NOT NULL UNIQUE,
  type_name VARCHAR(100) NOT NULL,
  code_prefix VARCHAR(20) NOT NULL,
  default_terms TEXT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 3. TABEL COUNTER PENOMORAN OTOMATIS (invoice_counters)
CREATE TABLE IF NOT EXISTS invoice_counters (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_type_id INT NOT NULL,
  year_month VARCHAR(6) NOT NULL, -- Format YYMM (contoh: 2608)
  last_sequence INT DEFAULT 0,
  CONSTRAINT fk_counter_type FOREIGN KEY (invoice_type_id) REFERENCES invoice_types(id) ON DELETE CASCADE,
  UNIQUE KEY uk_type_period (invoice_type_id, year_month)
) ENGINE=InnoDB;

-- 4. TABEL JENIS PRODUK & EVENT (item_types)
CREATE TABLE IF NOT EXISTS item_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_code VARCHAR(20) NOT NULL UNIQUE,
  category_name VARCHAR(100) NOT NULL,
  is_event BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 5. TABEL MASTER PRODUK & WORKSHOP (items)
CREATE TABLE IF NOT EXISTS items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_type_id INT NOT NULL,
  item_code VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(150) NOT NULL,
  unit_name VARCHAR(20) DEFAULT 'Pcs',
  price DECIMAL(12,2) NOT NULL,
  description TEXT NULL,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_items_type FOREIGN KEY (item_type_id) REFERENCES item_types(id)
) ENGINE=InnoDB;

-- 6. TABEL STOK & KUOTA TERPISAH (stocks)
```

```

CREATE TABLE IF NOT EXISTS stocks (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_id INT NOT NULL UNIQUE,
  qty_available INT NOT NULL DEFAULT 0,
  qty_sold INT NOT NULL DEFAULT 0,
  min_stock_alert INT DEFAULT 2,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_stocks_item FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 7. TABEL PELANGGAN (customers)
CREATE TABLE IF NOT EXISTS customers (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  email VARCHAR(100) NULL,
  address TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- 8. TABEL HEADER INVOICE (invoices)
CREATE TABLE IF NOT EXISTS invoices (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_number VARCHAR(50) NOT NULL UNIQUE,
  invoice_type_id INT NOT NULL,
  customer_id INT NOT NULL,
  issue_date DATE NOT NULL,
  due_date DATE NULL,
  subtotal DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  discount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  dp_amount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  grand_total DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  payment_status ENUM('LUNAS', 'DP', 'BELUM_LUNAS') NOT NULL DEFAULT 'BELUM_LUNAS',
  terms_and_notes TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_invoices_type FOREIGN KEY (invoice_type_id) REFERENCES invoice_types(id),
  CONSTRAINT fk_invoices_customer FOREIGN KEY (customer_id) REFERENCES customers(id)
) ENGINE=InnoDB;

-- 9. TABEL DETAIL BARANG INVOICE (invoice_items)
CREATE TABLE IF NOT EXISTS invoice_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT NOT NULL,
  item_id INT NULL,
  description VARCHAR(255) NOT NULL,
  qty INT NOT NULL DEFAULT 1,
  unit_name VARCHAR(20) DEFAULT 'Pcs',
  price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  total_price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  custom_notes TEXT NULL,
  CONSTRAINT fk_inv_items_invoice FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE,
  CONSTRAINT fk_inv_items_item FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 10. TABEL RIWAYAT PEMBAYARAN (payments)
CREATE TABLE IF NOT EXISTS payments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT NOT NULL,
  payment_date DATETIME NOT NULL,
  amount_paid DECIMAL(12,2) NOT NULL,
  payment_method VARCHAR(50) NOT NULL,
  proof_file VARCHAR(255) NULL,
  notes VARCHAR(255) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_payments_invoice FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- OPTIMASI INDEXING
CREATE INDEX idx_invoices_status ON invoices(payment_status);
CREATE INDEX idx_invoices_date ON invoices(issue_date);
CREATE INDEX idx_items_code ON items(item_code);
CREATE INDEX idx_customers_phone ON customers(phone);

```

4. Script SQL DML (Sample Data Sesuai Invoice Acuan)

Script ini melakukan pengisian data awal (*seeding*) termasuk contoh transaksi invoice DNB-WSP-2608-0003:

```

-- 1. PROFIL TOKO DYNOBOO
INSERT INTO company_profiles (company_name, tagline, phone_number, email, address)

```

```

VALUES ('DynoBoo', 'Handmade Crochet Dolls & Beaded Accessories', '0812-3456-7890', 'admin@dynoboo.com', 'Pontianak',

-- 2. MASTER TIPE INVOICE & SK
INSERT INTO invoice_types (type_code, type_name, code_prefix, default_terms)
VALUES
('WSP', 'Invoice Workshop', 'DNB-WSP', 'Dengan melakukan pembayaran, peserta dianggap telah membaca dan menyetujui s
('BQT', 'Invoice Bouquet', 'DNB-BQT', 'Harap melakukan pelunasan sebelum pesanan bouquet dikirimkan.'),
('RJT', 'Invoice Handicraft Rajut', 'DNB-RJT', 'Barang rajutan kustom membutuhkan waktu pembuatan 3-7 hari kerja.');
```

```

-- 3. JENIS PRODUK & EVENT
INSERT INTO item_types (category_code, category_name, is_event)
VALUES
('CAT-WKS', 'Event Workshop Pelatihan', TRUE),
('CAT-RJT', 'Produk Rajut Handicraft', FALSE),
('CAT-BQT', 'Bunga Bouquet Rajut', FALSE);

-- 4. KATALOG MASTER ITEM
INSERT INTO items (item_type_id, item_code, name, unit_name, price, description)
VALUES
(1, 'WKS-POT-01', 'Workshop Animal Pot', 'Pcs', 100000.00, 'Pelatihan membuat rajutan pot karakter hewan termasuk ba
(2, 'AMIG-DNO-01', 'Amigurumi Dino Pink', 'Pcs', 85000.00, 'Boneka rajut gantungan karakter dino pink'),
(3, 'BQT-ROSE-01', 'Bouquet Bunga Mawar Rajut', 'Set', 150000.00, 'Rangkaian 5 tangkai bunga mawar rajut');
```

```

-- 5. INITIALIZE STOK & KUOTA
INSERT INTO stocks (item_id, qty_available, qty_sold, min_stock_alert)
VALUES
(1, 15, 1, 2), -- Kuota tersisa 15 kursi
(2, 10, 5, 2),
(3, 8, 2, 1);

-- 6. DATA PELANGGAN
INSERT INTO customers (name, phone, email, address)
VALUES ('Nama Konsumen', '089876543210', 'konsumen@gmail.com', 'Pontianak City');
```

```

-- 7. INPUT TRANSAKSI INVOICE (Sesuai Contoh Gambar)
INSERT INTO invoices (invoice_number, invoice_type_id, customer_id, issue_date, subtotal, discount, dp_amount, grand
VALUES (
'DNB-WSP-2608-0003',
1,
1,
'2026-07-28',
100000.00,
0.00,
0.00,
100000.00,
'LUNAS',
'Dengan melakukan pembayaran, peserta dianggap telah membaca dan menyetujui seluruh syarat dan ketentuan worksho
);

-- 8. DETAIL ITEM DALAM INVOICE
INSERT INTO invoice_items (invoice_id, item_id, description, qty, unit_name, price, total_price)
VALUES (1, 1, 'Workshop Animal Pot', 1, 'Pcs', 100000.00, 100000.00);

-- 9. RIWAYAT PEMBAYARAN LUNAS
INSERT INTO payments (invoice_id, payment_date, amount_paid, payment_method, notes)
VALUES (1, '2026-07-28 10:15:00', 100000.00, 'Transfer BCA', 'Pelunasan Pendaftaran Workshop');
```

```

-- 10. UPDATE COUNTER PENOMORAN
INSERT INTO invoice_counters (invoice_type_id, year_month, last_sequence)
VALUES (1, '2608', 3)
ON DUPLICATE KEY UPDATE last_sequence = 3;
```

5. Cara Kerja Alur Sistem POS Web DynoBoo

- 1. Auto Generator Nomor Invoice:** Saat Admin membuat invoice baru dan memilih Tipe Transaksi (*misal: Workshop*), sistem membaca tabel `invoice_counters` untuk kombinasi WSP + 2608. Angka urutan terakhir (2) otomatis bertambah 1 menjadi 3, lalu dirangkai menjadi DNB-WSP-2608-0003.
- 2. Pemotongan Stok & Kuota Otomatis:** Ketika transaksi diproses, sistem otomatis mengurangi `qty_available` di tabel `stocks` dan menambah `qty_sold` sesuai dengan item yang ada di `invoice_items`.
- 3. Kalkulasi Status Indikator Pembayaran (Badge Warna):**
 - **LUNAS (Hijau):** Jika total entri pembayaran di `payments` $\geq$ `grand_total`.
 - **DP (Kuning):** Jika total entri pembayaran > 0 tetapi $<$ `grand_total`.
 - **BELUM LUNAS (Merah):** Jika belum ada entri pembayaran (total bayar = 0).

User prompt: Table invoice_types => code_prefix buat apa Table customer tidak perlu Table invoices ==> ditambahkan invoices_decription Jadi nanti dari table ini itu dia menyediakan template. Misalnya aku buat dua nih, satu table workshop, satu table produk. Jadi tinggal diklik nanti itu udah ada template-nya. Jadi tinggal mereka input harga, input nama pembeli aja, sama status payment-nya aja gitu. Jadi itu enggak perlu sampai terms and note, yang lain-lainnya lah. Itu tuh udah langsung auto generate gitu. Itu kurang lebih tuh bikin template buat ininya. Sama auto generate itu udah muncul juga. Jadi itu udah enak. Jadi mereka tinggal dari template itu tinggal klik klik klik aja. Setelah itu kan bisa menjadi invoice type kan juga bisa membantu untuk rekapan history penjualan. Jadi nanti bisa history penjualan bisa all, bisa per invoice type-nya yang misalnya workshop atau produk gitu.

Response: Penyesuaian ini membuat alur pembuatan invoice menjadi **jauh lebih cepat dan praktis** untuk operasional harian DynoBoo.

Berikut penjelasan mengenai fungsi code_prefix, konsep *template klik-klik*, serta pembaruan skema database terbarunya.

1. Fungsi code_prefix pada Tabel invoice_types

code_prefix berfungsi sebagai **awalan unik penomoran otomatis** untuk membedakan identitas invoice berdasarkan jenis transaksinya.

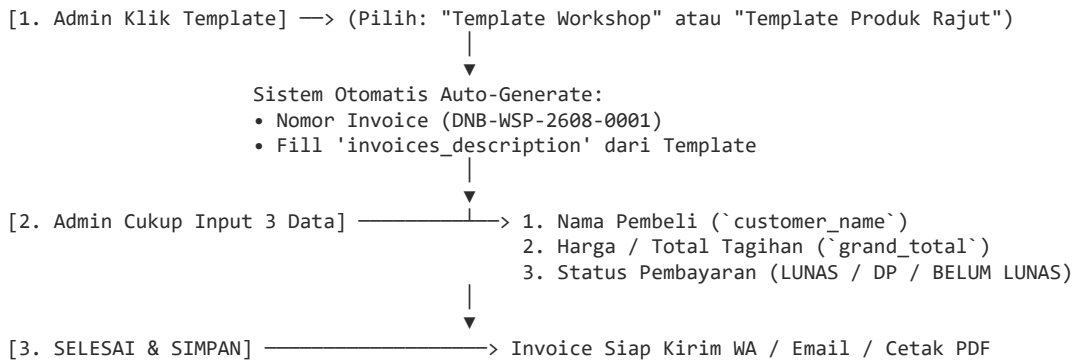
- Jika Admin memilih **Template Workshop**, code_prefix-nya adalah DNB-WSP. Nomor otomatis yang terbentuk: **DNB-WSP-2608-0001**.
- Jika Admin memilih **Template Produk Rajut**, code_prefix-nya adalah DNB-PRD. Nomor otomatis yang terbentuk: **DNB-PRD-2608-0001**.

Kegunaan Utama:

1. Memudahkan Admin mengenali jenis transaksi hanya dengan melihat nomor invoice-nya.
2. Memudahkan pencarian dan filter laporan history penjualan per jenis transaksi di kemudian hari.

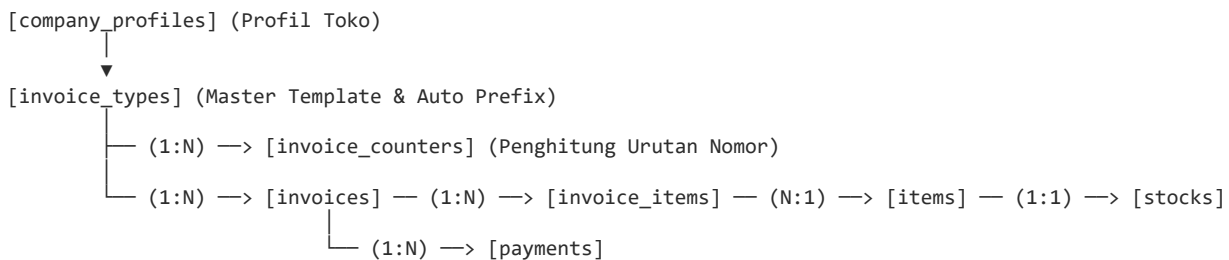
2. Konsep Alur Baru: "Click-&Fill Template" (Super Cepat)

Dengan menghapus tabel customer dan memanfaatkan invoice_types sebagai **Master Template**, alur kerja Admin saat buat invoice menjadi sangat ringkas:



3. Pembaruan Struktur Database Ter-Simplikasi

Tabel customers **resmi dihapus**. Kolom customer_name langsung ditaruh di dalam tabel invoices.



Penyesuaian Kolom Utama:

1. **Tabel invoice_types (Master Template):**
 - Menampung template_description bawaan yang otomatis tercopy saat invoice baru dibuat.
2. **Tabel invoices (Header Transaksi):**
 - customer_id diganti menjadi customer_name (Langsung ketik nama pembeli).
 - Ditambahkan invoices_description (Terisi otomatis dari template, bisa disesuaikan jika perlu).
 - Menampung payment_status (● Belum Lunas, ● DP, ● Lunas).

4. Script SQL DDL Terupdate (Create Tables)

```

CREATE DATABASE IF NOT EXISTS dynoboo_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE dynoboo_db;

-- 1. PROFIL TOKO (company_profiles)
CREATE TABLE IF NOT EXISTS company_profiles (
  id INT AUTO_INCREMENT PRIMARY KEY,
  company_name VARCHAR(100) NOT NULL,
  tagline VARCHAR(150) NULL,
  logo_path VARCHAR(255) NULL,
  phone_number VARCHAR(20) NOT NULL,
  email VARCHAR(100) NULL,
  address TEXT NULL
) ENGINE=InnoDB;

-- 2. MASTER TYPE / TEMPLATE INVOICE (invoice_types)
CREATE TABLE IF NOT EXISTS invoice_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  type_code VARCHAR(10) NOT NULL UNIQUE,      -- Contoh: WSP, PRD, BQT
  type_name VARCHAR(100) NOT NULL,            -- Contoh: Template Workshop, Template Produk Rajut
  code_prefix VARCHAR(20) NOT NULL,           -- Contoh: DNB-WSP, DNB-PRD
  template_description TEXT NULL,             -- Deskripsi/Catatan bawaan template
  is_active BOOLEAN DEFAULT TRUE
) ENGINE=InnoDB;

-- 3. COUNTER NOMOR URUT OTOMATIS (invoice_counters)
CREATE TABLE IF NOT EXISTS invoice_counters (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_type_id INT NOT NULL,
  year_month VARCHAR(6) NOT NULL,             -- Format YYMM (contoh: 2608)
  last_sequence INT DEFAULT 0,
  CONSTRAINT fk_counter_type FOREIGN KEY (invoice_type_id) REFERENCES invoice_types(id) ON DELETE CASCADE,
  UNIQUE KEY uk_type_period (invoice_type_id, year_month)
) ENGINE=InnoDB;

-- 4. JENIS PRODUK (item_types)
CREATE TABLE IF NOT EXISTS item_types (
  id INT AUTO_INCREMENT PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL,        -- 'Workshop', 'Produk Rajut', 'Bouquet'
  is_event BOOLEAN DEFAULT FALSE
) ENGINE=InnoDB;

-- 5. MASTER KATALOG PRODUK / EVENT (items)
CREATE TABLE IF NOT EXISTS items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_type_id INT NOT NULL,
  item_code VARCHAR(50) NOT NULL UNIQUE,
  name VARCHAR(150) NOT NULL,
  unit_name VARCHAR(20) DEFAULT 'Pcs',
  price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  CONSTRAINT fk_items_type FOREIGN KEY (item_type_id) REFERENCES item_types(id)
) ENGINE=InnoDB;

-- 6. STOK BARANG & KUOTA WORKSHOP (stocks)
CREATE TABLE IF NOT EXISTS stocks (
  id INT AUTO_INCREMENT PRIMARY KEY,
  item_id INT NOT NULL UNIQUE,
  qty_available INT NOT NULL DEFAULT 0,       -- Stok/Kuota sisa
  qty_sold INT NOT NULL DEFAULT 0,           -- Total terjual
  CONSTRAINT fk_stocks_item FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- 7. HEADER TRANSAKSI INVOICE (invoices) - REVISI TANPA CUSTOMER TABLE
CREATE TABLE IF NOT EXISTS invoices (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_number VARCHAR(50) NOT NULL UNIQUE, -- Auto Generate (misal: DNB-WSP-2608-0003)
  invoice_type_id INT NOT NULL,               -- Relasi ke Template (Untuk Filter History)
  customer_name VARCHAR(100) NOT NULL,        -- Input Nama Pembeli
  invoices_description TEXT NULL,             -- Auto fill dari template_description
  issue_date DATE NOT NULL,
  subtotal DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  discount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  dp_amount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  grand_total DECIMAL(12,2) NOT NULL DEFAULT 0.00, -- Input / Auto Calculate Total Harga
  payment_status ENUM('LUNAS', 'DP', 'BELUM_LUNAS') NOT NULL DEFAULT 'BELUM_LUNAS',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_invoices_type FOREIGN KEY (invoice_type_id) REFERENCES invoice_types(id)
) ENGINE=InnoDB;

-- 8. DETAIL ITEM INVOICE (invoice_items)

```

```

CREATE TABLE IF NOT EXISTS invoice_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT NOT NULL,
  item_id INT NULL,
  description VARCHAR(255) NOT NULL,
  qty INT NOT NULL DEFAULT 1,
  unit_name VARCHAR(20) DEFAULT 'Pcs',
  price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  total_price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  CONSTRAINT fk_inv_items_invoice FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE,
  CONSTRAINT fk_inv_items_item FOREIGN KEY (item_id) REFERENCES items(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- 9. RIWAYAT PEMBAYARAN (payments)
CREATE TABLE IF NOT EXISTS payments (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT NOT NULL,
  payment_date DATETIME NOT NULL,
  amount_paid DECIMAL(12,2) NOT NULL,
  payment_method VARCHAR(50) NOT NULL,          -- 'Transfer BCA', 'QRIS', 'Cash'
  notes VARCHAR(255) NULL,
  CONSTRAINT fk_payments_invoice FOREIGN KEY (invoice_id) REFERENCES invoices(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- INDEX UNTUK FILTER REKAPAN / HISTORY PENJUALAN
CREATE INDEX idx_invoices_type ON invoices(invoice_type_id);
CREATE INDEX idx_invoices_status ON invoices(payment_status);
CREATE INDEX idx_invoices_date ON invoices(issue_date);

```

5. Script SQL DML (Contoh Data Template & Simulasi Filter Rekap)

A. Membuatkan Master Template Transaksi

```

-- Membuatkan Template Invoice
INSERT INTO invoice_types (type_code, type_name, code_prefix, template_description)
VALUES
(
  'WSP',
  'Template Workshop',
  'DNB-WSP',
  'Dengan melakukan pembayaran, peserta dianggap telah membaca dan menyetujui seluruh syarat dan ketentuan worksho
),
(
  'PRD',
  'Template Produk Rajut',
  'DNB-PRD',
  'Pesanan rajutan kustom diproses setelah konfirmasi DP/Pelunasan. Estimasi pengerjaan 3-7 hari kerja.'
);

```

B. Simulasi Input Invoice Cepat (Hanya Input Nama, Total Harga & Status)

```

-- Contoh 1: Buat Invoice dari Template Workshop
INSERT INTO invoices (invoice_number, invoice_type_id, customer_name, invoices_description, issue_date, grand_total,
VALUES (
  'DNB-WSP-2608-0003',
  1,                      -- Dipilih: Template Workshop (ID 1)
  'Nama Konsumen',       -- Input Manual Nama
  (SELECT template_description FROM invoice_types WHERE id = 1), -- Auto Copied dari Template!
  '2026-07-28',
  100000.00,             -- Input Total Harga
  'LUNAS'                -- Input Status Pembayaran
);

-- Contoh 2: Detail Item (Auto terisi sesuai pilihan)
INSERT INTO invoice_items (invoice_id, description, qty, unit_name, price, total_price)
VALUES (1, 'Workshop Animal Pot', 1, 'Pcs', 100000.00, 100000.00);

```

C. Query Rekap History Penjualan (Sesuai Permintaan)

1. Tampilkan Semua History Penjualan (ALL):

```

SELECT
  i.invoice_number,
  t.type_name AS template_type,
  i.customer_name,
  i.issue_date,
  i.grand_total,
  i.payment_status

```

```
FROM invoices i
JOIN invoice_types t ON i.invoice_type_id = t.id
ORDER BY i.issue_date DESC;
```

2. Tampilkan History Penjualan Khusus WORKSHOP saja:

```
SELECT
    i.invoice_number,
    i.customer_name,
    i.issue_date,
    i.grand_total,
    i.payment_status
FROM invoices i
WHERE i.invoice_type_id = 1; -- ID 1 = Template Workshop
```

3. Tampilkan History Penjualan Khusus PRODUK RAJUT saja:

```
SELECT
    i.invoice_number,
    i.customer_name,
    i.issue_date,
    i.grand_total,
    i.payment_status
FROM invoices i
WHERE i.invoice_type_id = 2; -- ID 2 = Template Produk Rajut
```

Dengan skema database ter-update ini, pembuatan invoice di web admin DynoBoo dijamin jauh lebih efisien, dan laporan history penjualan bisa ditarik secara fleksibel per kategori!

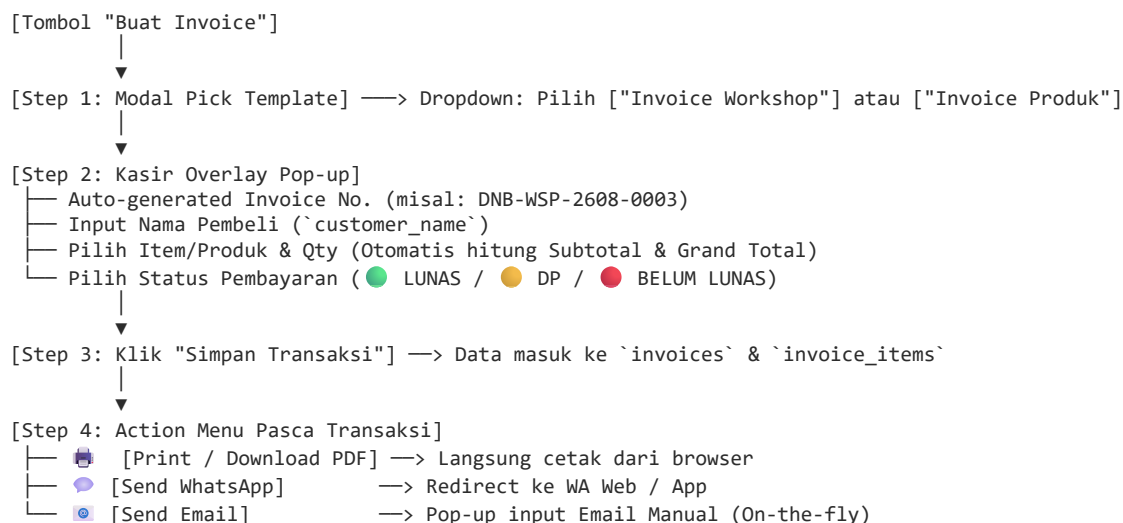
User prompt: Enggak perlu, tetap perlu template invoice. Cuma nanti pas UI-nya itu konteksnya beda. Nanti semisal user masuk ke konteks pembuatan invoice, nanti pas diklik disuruh pilih mau invoice mana, itu tinggal drag and drop button aja, eh drag and drop pula, drop down button aja. Jadi misalnya mereka tuh invoice typenya mereka inputnya dua misalnya, itu contoh workshop sama produk. Misalnya nanti pas invoice nanti muncul dua juga workshop atau produk. Jadi semisal mereka ketik produk, next, nanti mereka tuh tinggal isi customer sama harga belinya apa. Jadi langsung kayak buka kayak UI kasir gitu, tinggal di-overlay aja sih, nanti UI kasir gitu. Dari UI kasir itu, nanti mereka udah selesai isi identitasnya, dah, selesai, langsung kesimpan history. Jadi bisa mereka cetak, bisa mereka print PDF, bisa mereka send ke email juga. Oh iya, mau send email ya? Send email tak usah simpan dekat database ya? Nanti palingan bikinkan kayak menu send email, nanti kalau mereka klik, nanti muncul email. Jadi minta mereka input email secara manual baru ke-send aja gitu.

Response: Konsep alur UI/UX **Overlay Kasir (POS) dengan Dropdown Template** ini sangat praktis. Admin tidak akan pusing melihat halaman form yang panjang dan rumit. Cukup pilih tipe dari *dropdown*, lalu pop-up kasir muncul, isi data kilat, dan transaksi langsung tersimpan.

Mengenai **fitur Send Email tanpa simpan di database**, itu keputusan yang sangat tepat untuk efisiensi penyimpanan (*lightweight DB*).

1. Alur UI/UX Kasir Overlay & Eksekusi Invoice

Berikut gambaran alur pengerjaan di sisi *Frontend* dan *Backend*:



2. Cara Kerja Fitur "Send Email Manual" (Tanpa Simpan DB)

Karena email tidak disimpan di tabel database, sistem backend bekerja secara *on-demand*:

- Admin klik tombol **"Kirim Email"** di halaman riwayat/detail invoice.
- Muncul **Pop-up Form Kecil: "Masukkan Email Penerima"**.
- Admin mengetik email pembeli (contoh: pembeli@gmail.com) lalu klik **Kirim**.

4. Backend Action:

- Backend mengambil data transaksi dari `invoices` berdasarkan ID.
 - Backend me-render PDF Invoice secara *in-memory* (sementara di RAM server).
 - Backend mengirim email via SMTP / Email Service (seperti Resend atau Nodemailer) dengan lampiran PDF tersebut.
5. Setelah terkirim, pop-up ditutup dan sistem menampilkan notifikasi *"Email Berhasil Terkirim!"*. **Tidak ada data email yang tertinggal di database.**

3. Rekapitulasi Final Struktur Database DynoBoo (Super Ringkas)

Struktur ini sudah sangat efisien untuk mendukung UI Kasir Overlay dan laporan history penjualan:

A. Tabel `invoice_types` (Master Template)

Field Name	Type	Description
<code>id</code> (PK)	INT	ID Template
<code>type_code</code>	VARCHAR(10)	Code: WSP, PRD
<code>type_name</code>	VARCHAR(100)	Nama: "Invoice Workshop", "Invoice Produk"
<code>code_prefix</code>	VARCHAR(20)	Prefix Auto-number: DNB-WSP, DNB-PRD
<code>template_description</code>	TEXT	S&K Bawaan Template (Auto Fill)

B. Tabel `invoices` (Header Transaksi Kasir)

Field Name	Type	Description
<code>id</code> (PK)	INT	ID Invoice
<code>invoice_number</code>	VARCHAR(50)	Auto Generate: DNB-WSP-2608-0003
<code>invoice_type_id</code> (FK)	INT	Relasi ke Template (Untuk Filter History)
<code>customer_name</code>	VARCHAR(100)	Nama Pembeli (Diisi dari Kasir Overlay)
<code>invoices_description</code>	TEXT	Auto Copy dari <code>template_description</code>
<code>issue_date</code>	DATE	Tanggal Transaksi
<code>grand_total</code>	DECIMAL(12,2)	Total Tagihan Pembelian
<code>payment_status</code>	ENUM	'LUNAS' (Hijau), 'DP' (Kuning), 'BELUM_LUNAS' (Merah)

C. Tabel `invoice_items` (Detail Rincian Barang)

Field Name	Type	Description
<code>id</code> (PK)	INT	ID Item Transaksi
<code>invoice_id</code> (FK)	INT	Relasi ke <code>invoices.id</code>
<code>item_id</code> (FK)	INT	Relasi ke Master Katalog Produk/Workshop
<code>description</code>	VARCHAR(255)	Nama Produk / Workshop
<code>qty</code>	INT	Jumlah Dibeli
<code>price</code>	DECIMAL(12,2)	Harga Satuan
<code>total_price</code>	DECIMAL(12,2)	(<code>qty * price</code>)

4. Keuntungan Konsep Ini Bagi DynoBoo:

- Sangat Cepat saat Transaksi:** Admin hanya perlu **2-3 kali klik** di Kasir Overlay untuk menyelesaikan 1 invoice.
- Database Bersih:** Tidak ada tumpukan data email/alamat pelanggan yang memenuhi penyimpanan server.
- Rekapan Penjualan Sangat Fleksibel:** Laporan riwayat transaksi bisa difilter per jenis (*Semua Transaksi*, *Khusus Workshop*, atau *Khusus Produk Rajut*) hanya dengan memfilter `invoice_type_id`.

Apakah Anda ingin kita bahas contoh struktur *JSON Payload* / API Endpoint untuk interaksi Kasir Overlay ini ke Backend, atau mau langsung bahas rekomendasi *tech stack* (misalnya Laravel/Node.js/React) yang cocok untuk membangun website ini?

User prompt: oke skrh bantu buat kan konsep nya di gdocs atau pdf langsung untuk UI, Database dan konsep Alur nya macam flow kerjanya gitu. Nanti dari data yang kamu buat, nanti aku mau pakai agent AI untuk membuat kodenya. By the way, database-nya ini aku pakai Supabase serta nanti pakai Next.js untuk front-end-nya. Oke, dan untuk back-end nanti Next.js. Habis itu nanti di web service-nya itu lewat Vercel.

Response: Pilihan stack yang kamu ambil (**Next.js + Supabase + Vercel**) adalah salah satu *tech stack* modern paling efisien untuk dikerjakan bersama AI coding agent (seperti Cursor, Claude Code, atau v0). AI agent sangat paham dengan *pattern* dari stack ini.

Berikut adalah **Dokumen Cetak Biru (Architecture Blueprint)** lengkap dan terstruktur. Kamu bisa langsung menyalin (*copy-paste*) seluruh teks di bawah ini ke **Google Docs** atau menyimpannya sebagai **PDF**.

SPESIFIKASI SISTEM & BLUEPRINT ARSITEKTUR

Target Stack:

- **Frontend & Backend:** Next.js (App Router, Server Actions/Route Handlers, Tailwind CSS, shadcn/ui)
- **Database & Auth:** Supabase (PostgreSQL, Row Level Security / RLS, Auth SSR)
- **Hosting/Deployment:** Vercel

1. Arsitektur Folder & Struktur Aplikasi (Next.js App Router)

app/	
(auth)/	# Group route untuk autentikasi
login/page.tsx	
register/page.tsx	
/dashboard)/	# Group route terlindungi (butuh login)
layout.tsx	# Sidebar & Header
page.tsx	# Main Dashboard Summary
data/	
page.tsx	# Manajemen Input Data / Alternatif / Kriteria
process/	
page.tsx	# Halaman Proses/Kalkulasi Data
results/	
page.tsx	# Halaman Laporan & Hasil Akhir
api/	# Webhooks atau Custom API Routes (jika ada)
layout.tsx	# Root Layout & Font Setup
page.tsx	# Landing Page Utama
components/	
ui/	# Komponen Atom (Button, Input, Card dari shadcn/ui)
shared/	# Sidebar, Navbar, Footer
modules/	# Komponen spesifik fitur (DataForm, ResultTable, Chart)
lib/	
supabase/	
client.ts	# Client Component Supabase Client
server.ts	# Server Component/Action Supabase Client
middleware.ts	# Session refresh & Route protection
utils.ts	# Helper functions (math, formatting)
types/	
index.ts	# TypeScript definitions (database & app)
middleware.ts	# Protection guard untuk route dashboard

2. Skema Database Supabase (PostgreSQL + RLS)

Jalankan skrip SQL ini langsung di **Supabase SQL Editor** untuk membuat tabel, relasi, dan keamanan data berbasis pengguna (*Row Level Security*).

```
-- 1. Tabel Profil Pengguna (Tersinkronisasi dengan Supabase Auth)
CREATE TABLE public.profiles (
  id UUID REFERENCES auth.users ON DELETE CASCADE PRIMARY KEY,
  full_name TEXT,
  role TEXT DEFAULT 'user' CHECK (role IN ('admin', 'user')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

-- Turn on RLS for profiles
ALTER TABLE public.profiles ENABLE ROW LEVEL SECURITY;

CREATE POLICY "Pengguna dapat melihat profil sendiri"
ON public.profiles FOR SELECT
USING (auth.uid() = id);

CREATE POLICY "Pengguna dapat memperbarui profil sendiri"
```

```

ON public.profiles FOR UPDATE
USING (auth.uid() = id);

-- 2. Tabel Proyek / Kasus Keputusan (Projects / Datasets)
CREATE TABLE public.projects (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  title VARCHAR(255) NOT NULL,
  description TEXT,
  status VARCHAR(50) DEFAULT 'draft' CHECK (status IN ('draft', 'processed', 'archived')),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL,
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

ALTER TABLE public.projects ENABLE ROW LEVEL SECURITY;

CREATE POLICY "User hanya bisa mengelola proyek miliknya"
ON public.projects FOR ALL
USING (auth.uid() = user_id);

-- 3. Tabel Kriteria Evaluation (Criteria)
CREATE TABLE public.criteria (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  project_id UUID REFERENCES public.projects(id) ON DELETE CASCADE NOT NULL,
  code VARCHAR(50) NOT NULL,          -- Contoh: C1, C2
  name VARCHAR(255) NOT NULL,         -- Contoh: Biaya, Kualitas
  weight NUMERIC(5,4) NOT NULL,       -- Bobot kriteria (misal: 0.25)
  type VARCHAR(10) CHECK (type IN ('benefit', 'cost')) NOT NULL,
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

ALTER TABLE public.criteria ENABLE ROW LEVEL SECURITY;

CREATE POLICY "User mengelola kriteria via proyeknya"
ON public.criteria FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM public.projects
    WHERE projects.id = criteria.project_id AND projects.user_id = auth.uid()
  )
);

-- 4. Tabel Alternatif Data (Alternatives)
CREATE TABLE public.alternatives (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  project_id UUID REFERENCES public.projects(id) ON DELETE CASCADE NOT NULL,
  code VARCHAR(50) NOT NULL,          -- Contoh: A1, A2
  name VARCHAR(255) NOT NULL,         -- Contoh: Opsi A, Kandidaat B
  created_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

ALTER TABLE public.alternatives ENABLE ROW LEVEL SECURITY;

CREATE POLICY "User mengelola alternatif via proyeknya"
ON public.alternatives FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM public.projects
    WHERE projects.id = alternatives.project_id AND projects.user_id = auth.uid()
  )
);

-- 5. Tabel Nilai Matriks Evaluasi (Evaluation Values)
CREATE TABLE public.evaluations (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  alternative_id UUID REFERENCES public.alternatives(id) ON DELETE CASCADE NOT NULL,
  criterion_id UUID REFERENCES public.criteria(id) ON DELETE CASCADE NOT NULL,
  value NUMERIC(10,4) NOT NULL,
  UNIQUE(alternative_id, criterion_id)
);

ALTER TABLE public.evaluations ENABLE ROW LEVEL SECURITY;

CREATE POLICY "User mengelola matriks evaluasi miliknya"
ON public.evaluations FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM public.alternatives
    JOIN public.projects ON projects.id = alternatives.project_id

```

```
WHERE alternatives.id = evaluations.alternative_id AND projects.user_id = auth.uid()
)
);

-- 6. Tabel Hasil Perhitungan (Calculation Results)
CREATE TABLE public.results (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  project_id UUID REFERENCES public.projects(id) ON DELETE CASCADE NOT NULL,
  alternative_id UUID REFERENCES public.alternatives(id) ON DELETE CASCADE NOT NULL,
  score NUMERIC(10,6) NOT NULL,
  rank INT NOT NULL,
  calculated_at TIMESTAMP WITH TIME ZONE DEFAULT TIMEZONE('utc'::text, NOW()) NOT NULL
);

ALTER TABLE public.results ENABLE ROW LEVEL SECURITY;

CREATE POLICY "User melihat hasil kalkulasi miliknya"
ON public.results FOR ALL
USING (
  EXISTS (
    SELECT 1 FROM public.projects
    WHERE projects.id = results.project_id AND projects.user_id = auth.uid()
  )
);
```

3. Konsep UI / UX & Desain Halaman

Halaman	Deskripsi & Komponen Utama	Akses Kontrol
Landing Page (/)	Hero Section, penjelasan fitur utama, CTA Login/Register.	Publik
Auth (/login, /register)	Form login/daftar dengan email + password atau OAuth (Google).	Publik
Dashboard (/page.tsx)	Metrics ringkasan (Jumlah Proyek, Status Analisis), list proyek terbaru.	Authenticated
Data Input (/data)	Tabbed Interface: 1. Input Kriteria & Bobot, 2. Input Alternatif, 3. Matriks Nilai (Tabel Interaktif).	Authenticated
Proses Analisis (/process)	Tombol "Jalankan Kalkulasi", animasi loading indikator proses, visualisasi matriks normalisasi.	Authenticated
Hasil & Laporan (/results)	Table ranking akhir, chart visualisasi (Bar/Radar Chart), tombol Ekspor PDF / Excel.	Authenticated

4. Alur Kerja Sistem & Data Flow

A. Alur Autentikasi Pengguna

- User Sign Up/Login** -> Mengisi form di Next.js Frontend.
- Supabase Auth** memverifikasi kredensial dan mengembalikan **JWT Session Cookie**.
- Next.js Middleware** membaca cookie di server side:
 - Jika tidak valid -> Redirect ke /login.
 - Jika valid -> Diizinkan masuk ke rute /dashboard/*.

B. Alur Pengolahan & Kalkulasi Data (Server Action Pattern)

- Input Data:** Pengguna mengisi kriteria, bobot, dan alternatif pada tabel di /data.
- Submit via Server Action:** Frontend memanggil Next.js Server Action untuk menyimpan data batch ke Supabase (projects, criteria, alternatives, evaluations).
- Eksekusi Perhitungan:**
 - Pengguna mengklik tombol *"Hitung / Analisis"*.
 - Backend Next.js mengambil data matriks evaluasi dari Supabase.
 - Algoritma dijalankan di Server Node.js (aman, tidak terekspos di browser client).
 - Hasil kalkulasi (skor & *ranking*) disimpan kembali ke tabel results.
- Output:** Frontend menampilkan hasil peringkat, grafik chart, serta memberikan opsi ekspor laporan (PDF).

5. Panduan Konfigurasi Vercel (Deployment)

1. **Environment Variables** yang wajib diisi di Vercel Settings:
 - NEXT_PUBLIC_SUPABASE_URL: (URL Supabase Project)
 - NEXT_PUBLIC_SUPABASE_ANON_KEY: (Public Anon Key)
 - SUPABASE_SERVICE_ROLE_KEY: (Key khusus backend server/admin)
2. **Build Settings:** Standard Next.js framework preset (npm run build).

💡 Master Prompt (Untuk Diberikan ke Agent AI / Cursor / Claude Code)

Salin teks di bawah ini saat kamu meminta Agent AI mulai membangun kodenya:

Halo Agent, buatlah aplikasi web utuh berdasarkan cetak biru berikut:

TECH STACK:
- Framework: Next.js (App Router, TypeScript)
- UI Library: Tailwind CSS, shadcn/ui, Lucide Icons
- Database & Auth: Supabase PostgreSQL, @supabase/ssr
- Deployment: Vercel

PETUNJUK IMPLEMENTASI:
1. Ikuti struktur folder Next.js App Router & Skema Database yang ada di dokumen spesifikasi.
2. Gunakan Server Actions untuk operasi mutasi data (CRUD) dan query Supabase dari Server Components jika memungkinkan.
3. Gunakan middleware.ts untuk proteksi rute terotentikasi di bawah path /(dashboard).
4. Buat komponen form data interaktif dengan validasi menggunakan React Hook Form & Zod.
5. Tampilkan tabel hasil kalkulasi secara rapi dengan Chart (Recharts) dan fitur ekspor data.

Silakan mulai buatlah struktur file dasar beserta lib/supabase client & middleware terlebih dahulu.

Apakah skema database dan alur kerja di atas sudah sesuai dengan metode atau domain aplikasi yang ingin kamu buat, atau ada tabel/fitur spesifik yang ingin disesuaikan sebelum di-export ke PDF?

User prompt: *mana databasenya deskripsinya*

Response: Berikut adalah deskripsi lengkap seluruh struktur database **Supabase (PostgreSQL)** untuk website Invoice & POS **DynoBoo**, lengkap dengan fungsi tiap tabel, kamus data, dan script SQL DDL-nya.

1. Ringkasan Modul Database DynoBoo

Database ini terdiri dari **9 tabel utama** dan **1 Stored Function (RPC)** yang dirancang ter-normalisasi untuk kebutuhan kasir overlay POS, auto-numbering, dan laporan penjualan:

1. **company_profiles**: Identitas toko DynoBoo (logo, nama, kontak, alamat) untuk cetakan invoice.
2. **invoice_types**: Master template invoice (Workshop, Produk Rajut, dll) beserta prefix penomoran.
3. **invoice_counters**: Penyimpan urutan nomor invoice otomatis per bulan.
4. **item_types**: Kategori item (Workshop vs Produk Fisik).
5. **items**: Master katalog produk rajut dan tiket workshop.
6. **stocks**: Pengelola sisa stok/kuota (qty_available) dan jumlah terjual (qty_sold).
7. **invoices**: Header transaksi invoice (nama pembeli, total tagihan, status warna).
8. **invoice_items**: Detail rincian daftar barang yang dibeli per invoice (Parent-Child).
9. **payments**: Log riwayat pembayaran (DP / Pelunasan).

2. Kamus Data & Spesifikasi Tabel (Supabase PostgreSQL)

A. Modul Identitas & Template Transaksi

1. Tabel company_profiles (Profil Toko DynoBoo)

Menyimpan profil toko agar informasi di header invoice bersifat dinamis.

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Unik Profil
company_name	VARCHAR(100)	NOT NULL, DEFAULT 'DynoBoo'	Nama Toko
tagline	VARCHAR(150)	DEFAULT 'Handmade Crochet...'	Tagline Usaha

Field Name	Data Type	Constraints	Deskripsi / Contoh
logo_url	TEXT	NULL	Path / URL Logo Toko
phone_number	VARCHAR(20)	NOT NULL	Nomor WhatsApp Toko
email	VARCHAR(100)	NULL	Email Resmi Toko
address	TEXT	NULL	Alamat Studio Rajut

2. Tabel invoice_types (Master Template Invoice)

Menyimpan daftar template transaksi yang muncul di dropdown POS Overlay beserta syarat & ketentuan (S&K) bawaan.

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Template
type_code	VARCHAR(10)	UNIQUE, NOT NULL	Kode Singkat: 'WSP', 'PRD', 'BQT'
type_name	VARCHAR(100)	NOT NULL	Contoh: "Invoice Workshop", "Invoice Produk"
code_prefix	VARCHAR(20)	NOT NULL	Awalan Nomor: "DNB-WSP", "DNB-PRD"
template_description	TEXT	NULL	Template S&K bawaan yang otomatis dicopy ke invoice
is_active	BOOLEAN	DEFAULT TRUE	Status template aktif/nonaktif

3. Tabel invoice_counters (Counter Penomoran Otomatis)

Penghitung urutan transaksi atomic per jenis invoice dan per bulan/tahun (YYMM).

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Counter
invoice_type_id	BIGINT	FK -> invoice_types(id)	Relasi ke tipe invoice
year_month	VARCHAR(6)	NOT NULL	Format YYMM, contoh: '2608' (Agustus 2026)
last_sequence	INT	DEFAULT 0	Urutan terakhir (misal: 3 untuk DNB-WSP-2608-0003)

B. Modul Katalog & Stok

4. Tabel item_types (Jenis Produk / Event)

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Kategori
category_name	VARCHAR(100)	NOT NULL	"Workshop", "Produk Rajut", "Bouquet"
is_event	BOOLEAN	DEFAULT FALSE	TRUE jika event/workshop, FALSE jika barang

5. Tabel items (Master Katalog Produk & Workshop)

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Unik Item Master
item_type_id	BIGINT	FK -> item_types(id)	Relasi ke Kategori
item_code	VARCHAR(50)	UNIQUE, NOT NULL	Kode Item: 'WKS-POT-01', 'AMIG-DNO-01'
name	VARCHAR(150)	NOT NULL	"Workshop Animal Pot", "Amigurumi Dino"
unit_name	VARCHAR(20)	DEFAULT 'Pcs'	Satuan: 'Pcs', 'Tiket', 'Set'
price	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00	Harga satuan dasar
is_active	BOOLEAN	DEFAULT TRUE	Status aktif dijual

6. Tabel stocks (Saldo Stok & Kuota)

Terhubung 1:1 dengan items untuk mencatat kuota/stok sisa dan jumlah yang terjual.

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Stok
item_id	BIGINT	UNIQUE, FK -> items(id)	Relasi 1:1 ke tabel items
qty_available	INT	NOT NULL, DEFAULT 0	Sisa stok / sisa kuota workshop
qty_sold	INT	NOT NULL, DEFAULT 0	Accumulative total unit terjual

C. Modul Transaksi Invoice (Kasir Overlay & History)

7. Tabel invoices (Header Transaksi)

Menyimpan ringkasan transaksi, nama pembeli, status pembayaran (badge warna), dan hasil auto-numbering.

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Transaksi
invoice_number	VARCHAR(50)	UNIQUE, NOT NULL	Result dari RPC: 'DNB-WSP-2608-0003'
invoice_type_id	BIGINT	FK -> invoice_types(id)	Template yang dipilih
customer_name	VARCHAR(100)	NOT NULL	Nama Pembeli (Diisi langsung via Kasir Overlay)
invoices_description	TEXT	NULL	Auto fill dari template_description
issue_date	DATE	NOT NULL, DEFAULT CURRENT_DATE	Tanggal penerbitan invoice
subtotal	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00	Total harga item
discount	DECIMAL(12,2)	DEFAULT 0.00	Potongan harga
dp_amount	DECIMAL(12,2)	DEFAULT 0.00	Uang muka yang sudah diterima
grand_total	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00	Total tagihan akhir
payment_status	payment_status_enum	DEFAULT 'BELUM_LUNAS'	Status: 'LUNAS' (🟢), 'DP' (🟡), 'BELUM_LUNAS' (🔴)

8. Tabel invoice_items (Detail Barang Terjual)

Menampung banyak item barang/workshop dalam 1 invoice (Relasi 1:N ke invoices).

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Item Invoice
invoice_id	BIGINT	FK -> invoices(id) ON DELETE CASCADE	Relasi ke Parent Invoice
item_id	BIGINT	FK -> items(id) ON DELETE SET NULL	Relasi ke Katalog Item
description	VARCHAR(255)	NOT NULL	Nama produk / workshop saat transaksi
qty	INT	NOT NULL, DEFAULT 1	Jumlah unit/tiket dibeli
unit_name	VARCHAR(20)	DEFAULT 'Pcs'	Satuan item
price	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00	Harga satuan saat transaksi
total_price	DECIMAL(12,2)	NOT NULL, DEFAULT 0.00	Kalkulasi (qty * price)

9. Tabel payments (Riwayat Pembayaran)

Field Name	Data Type	Constraints	Deskripsi / Contoh
id	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	ID Pembayaran

Field Name	Data Type	Constraints	Deskripsi / Contoh
invoice_id	BIGINT	FK -> invoices(id) ON DELETE CASCADE	Relasi ke Invoice
payment_date	TIMESTAMPZ	NOT NULL, DEFAULT NOW()	Waktu uang masuk
amount_paid	DECIMAL(12,2)	NOT NULL	Nominal pembayaran (DP / Pelunasan)
payment_method	VARCHAR(50)	DEFAULT 'Transfer BCA'	Metode: Transfer, QRIS, Cash
notes	VARCHAR(255)	NULL	Catatan tambahan

3. Script SQL DDL & RPC Function (Copy-Paste ke Supabase)

Jalankan script SQL ini langsung pada **SQL Editor** di Dashboard Supabase milikmu:

```
-- =====
-- SUPABASE POSTGRES SQL SCHEMA – DYNOBOO POS & INVOICE SYSTEM
-- =====

-- 1. ENUM STATUS PEMBAYARAN
CREATE TYPE payment_status_enum AS ENUM ('LUNAS', 'DP', 'BELUM_LUNAS');

-- 2. TABEL PROFIL TOKO (company_profiles)
CREATE TABLE company_profiles (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  company_name VARCHAR(100) NOT NULL DEFAULT 'DynoBoo',
  tagline VARCHAR(150) DEFAULT 'Handmade Crochet Dolls & Beaded Accessories',
  logo_url TEXT NULL,
  phone_number VARCHAR(20) NOT NULL DEFAULT '0812-3456-7890',
  email VARCHAR(100) NULL,
  address TEXT NULL,
  updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 3. TABEL MASTER TEMPLATE INVOICE (invoice_types)
CREATE TABLE invoice_types (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  type_code VARCHAR(10) UNIQUE NOT NULL,          -- 'WSP', 'PRD', 'BQT'
  type_name VARCHAR(100) NOT NULL,                 -- 'Invoice Workshop', 'Invoice Produk'
  code_prefix VARCHAR(20) NOT NULL,                 -- 'DNB-WSP', 'DNB-PRD'
  template_description TEXT NULL,                   -- Auto Copy S&K Bawaan Template
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 4. TABEL COUNTER PENOMORAN OTOMATIS (invoice_counters)
CREATE TABLE invoice_counters (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  invoice_type_id BIGINT REFERENCES invoice_types(id) ON DELETE CASCADE,
  year_month VARCHAR(6) NOT NULL,                   -- Format YYYYMM (misal: '2608')
  last_sequence INT DEFAULT 0,
  CONSTRAINT uk_type_period UNIQUE (invoice_type_id, year_month)
);

-- 5. TABEL KATALOG JENIS PRODUK & EVENT (item_types)
CREATE TABLE item_types (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  category_name VARCHAR(100) NOT NULL,              -- 'Workshop', 'Produk Rajut'
  is_event BOOLEAN DEFAULT FALSE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 6. TABEL MASTER ITEM & WORKSHOP (items)
CREATE TABLE items (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  item_type_id BIGINT REFERENCES item_types(id) ON DELETE RESTRICT,
  item_code VARCHAR(50) UNIQUE NOT NULL,
  name VARCHAR(150) NOT NULL,
  unit_name VARCHAR(20) DEFAULT 'Pcs',
  price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 7. TABEL STOK & KUOTA (stocks)
CREATE TABLE stocks (
  id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
```

```

    item_id BIGINT UNIQUE REFERENCES items(id) ON DELETE CASCADE,
    qty_available INT NOT NULL DEFAULT 0,          -- Stok/Kuota Sisa
    qty_sold INT NOT NULL DEFAULT 0,              -- Total Terjual
    updated_at TIMESTAMPTZ DEFAULT NOW()
);

-- 8. TABEL HEADER TRANSAKSI INVOICE (invoices)
CREATE TABLE invoices (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    invoice_number VARCHAR(50) UNIQUE NOT NULL,    -- Hasil dari RPC
    invoice_type_id BIGINT REFERENCES invoice_types(id) ON DELETE RESTRICT,
    customer_name VARCHAR(100) NOT NULL,          -- Input dari Kasir Overlay
    invoices_description TEXT NULL,               -- Auto fill dari template_description
    issue_date DATE NOT NULL DEFAULT CURRENT_DATE,
    subtotal DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    discount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    dp_amount DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    grand_total DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    payment_status payment_status_enum NOT NULL DEFAULT 'BELUM_LUNAS',
    created_at TIMESTAMPTZ DEFAULT NOW()
);

-- 9. TABEL DETAIL ITEM INVOICE (invoice_items)
CREATE TABLE invoice_items (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    invoice_id BIGINT REFERENCES invoices(id) ON DELETE CASCADE,
    item_id BIGINT REFERENCES items(id) ON DELETE SET NULL,
    description VARCHAR(255) NOT NULL,
    qty INT NOT NULL DEFAULT 1,
    unit_name VARCHAR(20) DEFAULT 'Pcs',
    price DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    total_price DECIMAL(12,2) NOT NULL DEFAULT 0.00
);

-- 10. TABEL PEMBAYARAN (payments)
CREATE TABLE payments (
    id BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    invoice_id BIGINT REFERENCES invoices(id) ON DELETE CASCADE,
    payment_date TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    amount_paid DECIMAL(12,2) NOT NULL,
    payment_method VARCHAR(50) NOT NULL DEFAULT 'Transfer BCA',
    notes VARCHAR(255) NULL
);

-- INDEXING OPTIMIZATION
CREATE INDEX idx_invoices_type ON invoices(invoice_type_id);
CREATE INDEX idx_invoices_status ON invoices(payment_status);
CREATE INDEX idx_invoices_date ON invoices(issue_date);

-- =====
-- SUPABASE STORED FUNCTION (RPC): AUTO GENERATE INVOICE NUMBER ATOMIC
-- =====
CREATE OR REPLACE FUNCTION generate_invoice_number(p_invoice_type_id BIGINT)
RETURNS TEXT
LANGUAGE plpgsql
AS $$
DECLARE
    v_prefix VARCHAR(20);
    v_year_month VARCHAR(6);
    v_seq INT;
    v_result TEXT;
BEGIN
    -- 1. Ambil prefix dari invoice_types
    SELECT code_prefix INTO v_prefix FROM invoice_types WHERE id = p_invoice_type_id;
    IF v_prefix IS NULL THEN
        RAISE EXCEPTION 'Invoice type ID % not found', p_invoice_type_id;
    END IF;

    -- 2. Format YYYY (contoh: 2608)
    v_year_month := TO_CHAR(CURRENT_DATE, 'YYYY');

    -- 3. Upsert counter & dapatkan sequence terbaru secara atomic
    INSERT INTO invoice_counters (invoice_type_id, year_month, last_sequence)
    VALUES (p_invoice_type_id, v_year_month, 1)
    ON CONFLICT (invoice_type_id, year_month)
    DO UPDATE SET last_sequence = invoice_counters.last_sequence + 1
    RETURNING last_sequence INTO v_seq;

    -- 4. Format hasil akhir: DNB-WSP-2608-0001

```

```
v_result := v_prefix || '-' || v_year_month || '-' || LPAD(v_seq::TEXT, 4, '0');  
RETURN v_result;  
END;  
$$;
```
